Chapter 1
Some Basic Remarks

1.1 Motivation

Let us introduce the basic concepts and notations: Computational physics aims at solving physical problems by means of numerical methods developed in the field of numerical analysis. According to I. JACQUES and C. JUDD [1], numerical analysis is defined as, quote: "*Numerical analysis is concerned with the development and analysis of methods for the numerical solution of practical problems.*" Although the term *practical problems* remained unspecified in this definition, it is certainly necessary to reflect on ways to find approximate solutions to complex problems which occur regularly in natural sciences. In fact, in most cases it is not possible to find analytic solutions and one must rely on good approximations. Let us give some examples.

Consider the definite integral

$$\int_a^b \mathrm{d}x \exp\left(-x^2\right), \tag{1.1}$$

which, for instance, may occur when it is required to calculate the probability that an event following a normal distribution takes on a value within the interval $[a, b]$, where $a, b \in \mathbb{R}$. In contrast to the much simpler integral

$$\int_a^b \mathrm{d}x \exp(x) = \exp(b) - \exp(a), \tag{1.2}$$

the integral (1.1) cannot be solved analytically because there is no elementary function which differentiates to $\exp\left(-x^2\right)$. Hence, we have to approximate this integral in such a way that the approximation is accurate enough for our purpose. This example illustrates that even mathematical expressions which appear quite simple at the first

B. A. Stickler and E. Schachinger, *Basic Concepts in Computational Physics*,
DOI: 10.1007/978-3-319-02435-6_1, © Springer International Publishing Switzerland 2014

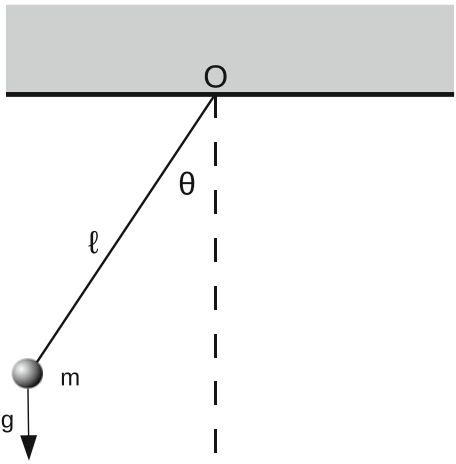


Fig. 1.1 Schematic illustration of the pendulum

glance may need a closer inspection when a numerical estimate for the expression is required. In fact, most numerical methods we will encounter within this book have been designed before the invention of modern computers or calculators. However, the applicability of these methods has increased and is still increasing drastically with the development of even more powerful machines. We give another example, namely the oscillation of a pendulum. From basic mechanics [2] we know that the time evolution of a frictionless pendulum of mass m and length ℓ in a gravitational field is defined by the differential equation

$$\ddot{\theta} + \frac{g}{\ell} \sin(\theta) = 0. \tag{1.3}$$

The solution of this equation describes the oscillatory motion of the pendulum around the origin O within a two-dimensional plane (Fig. 1.1). Here θ is the angular displacement and g is the acceleration due to gravity. Furthermore, we restrict the discussion to initial conditions of the form

$$\begin{cases} \theta(0) = \theta_0, \\ \dot{\theta}(0) = 0. \end{cases} \tag{1.4}$$

In the small angle approximation Eq. (1.3) is readily solved: For $\theta \ll 1$, in particular, $\theta_0 \ll 1$, we set $\sin(\theta) \approx \theta$ and obtain the differential equation of the harmonic oscillator:

$$\ddot{\theta} + \frac{g}{\ell}\theta = 0. \tag{1.5}$$

Together with the initial conditions (1.4) we arrive at the solution

$$\theta(t) = \theta_0 \cos(\omega t), \tag{1.6}$$

with

$$\omega = \sqrt{\frac{g}{\ell}}\,. \tag{1.7}$$

The period τ of the pendulum follows immediately:

$$\tau = 2\pi\sqrt{\frac{g}{\ell}}\,. \tag{1.8}$$

However, if the approximation of a small angular displacement $\theta_0 \ll 1$ is not applicable, expressions (1.6) and (1.8) will not be valid. Thus, it is advisable to apply energy conservation in order to arrive at analytic results. The energy is given by:

$$E = \frac{1}{2}mv^2 + mg\ell\,[1 - \cos(\theta)] = \frac{1}{2}mv_0^2 + mg\ell\,[1 - \cos(\theta_0)]. \tag{1.9}$$

Here v is the velocity of the point mass m and v_0 and θ_0 are defined by the initial conditions (1.4). Since $\dot{\theta}(0) = 0$ we have

$$\begin{aligned} E &= mg\ell\,[1 - \cos(\theta_0)] \\ &= 2mg\ell\sin^2\left(\frac{\theta_0}{2}\right), \end{aligned} \tag{1.10}$$

where we made use of:

$$1 - \cos(x) = 2\sin^2\left(\frac{x}{2}\right). \tag{1.11}$$

We use this result in Eq. (1.9) and arrive at:

$$\frac{1}{2}v^2 = 2g\ell\left[\sin^2\left(\frac{\theta_0}{2}\right) - \sin^2\left(\frac{\theta}{2}\right)\right]. \tag{1.12}$$

Since $v^2 = \ell^2\dot{\theta}^2$ we have

$$\dot{\theta} = 2\sqrt{\frac{g}{\ell}}\sqrt{\sin^2\left(\frac{\theta_0}{2}\right) - \sin^2\left(\frac{\theta}{2}\right)}. \tag{1.13}$$

Separation of variables yields

$$\sqrt{\frac{g}{\ell}}t = \frac{1}{2k}\int_0^{\theta} \frac{\mathrm{d}\varphi}{\sqrt{1 - \frac{1}{k^2}\sin^2\left(\frac{\varphi}{2}\right)}}, \tag{1.14}$$

with $k = \sin(\theta_0/2)$. For $t = \tau$ we have $\theta = \theta_0$ and we obtain for the period

$$\tau = \frac{2}{k}\sqrt{\frac{\ell}{g}} \int_0^{\theta_0} \frac{\mathrm{d}\varphi}{\sqrt{1 - \frac{1}{k^2}\sin^2\left(\frac{\varphi}{2}\right)}}, \tag{1.15}$$

where we defined the limits of integration as $\theta \in [-\theta_0, \theta_0]$. Let us transform the above integral into a more convenient form with help of the substitution $k\sin(\alpha) = \sin(\varphi/2)$. Thus, $\alpha \in [0, \pi/2]$ and a straightforward calculation yields:

$$\begin{aligned} \tau &= 4\sqrt{\frac{\ell}{g}} \int_0^{\frac{\pi}{2}} \frac{\mathrm{d}\alpha}{\sqrt{1 - k^2\sin^2(\alpha)}} \\ &= 4\sqrt{\frac{\ell}{g}} K_1(k). \end{aligned} \tag{1.16}$$

The function $K_1(k)$ introduced in (1.16) for $k \in \mathbb{R}$ is referred to as the complete elliptic integral of the first kind. All these manipulations did not really result in a simplification of the problem at hand because we are still confronted with the integral in Eq. (1.16) which cannot be evaluated without the use of additional approximations which will, in the end, result in a numerical solution of the problem. A natural way to proceed would be to expand the complete elliptic integral in a power series up to order N, where N is chosen in such a way that the *truncation error* $R_N(k)$ becomes negligible. We can find the desired expression in any text on special functions [3–5]. It reads

$$\begin{aligned} K_1(k) &= \frac{\pi}{2}\sum_{n=0}^{\infty}\left[\frac{(2n)!}{2^{2n}(n!)^2}\right]^2 k^{2n} \\ &= \frac{\pi}{2}\sum_{n=0}^{N}\left[\frac{(2n)!}{2^{2n}(n!)^2}\right]^2 k^{2n} + R_N(k). \end{aligned} \tag{1.17}$$

Furthermore, imagine the inverse problem: the period τ is given and the initial angle θ_0 is unknown. Again, we could expand the integrand in a power series and solve the corresponding polynomial for θ_0. However, such an approach would be very inefficient due to two reasons: first of all, we are confronted with the impossibility of finding analytically the roots of a polynomial of order $N > 4$ and, secondly, at which value of N should we truncate the power series if θ_0 is unknown? A glance in a book on special functions might give us a better, i.e. more convenient, alternative. Indeed, the inverse functions of the incomplete elliptic integral $F(a, b)$ with respect to a are referred to as JACOBI elliptic functions [3–5]. Given the equation

$$u = F(a, b), \tag{1.18}$$

the quantity of interest b can be expressed in terms of the JACOBI amplitude $\mathrm{am}(u, b)$, where

$$\mathrm{am}(u, a) = b. \tag{1.19}$$

Series expansions of $\mathrm{am}(u, a)$ have been developed, such that we can approximate θ_0 by truncating the respective series. These two examples illustrated that we have to depend on numerical approximations of definite expressions in a multitude of cases. Even if an approximate solution has been found for a particular problem it will be adamant to check quite carefully if the approach was (i) justified within the required accuracy, and (ii) if it allowed to improve the induced error of the result. The second point is known as the *stability* of a routine. We will discuss this topic in more detail in Sect. 1.4.

Throughout this book we will be confronted with numerous methods which will allow approximate solutions of problems similar to the two examples illustrated above. First of all, we would like to specify the properties we expect these methods to have. Primarily, the method is to be formulated as an unambiguous mathematical recipe which can be applied to the set of problems it was designed for. Its applicability should be well defined and it should allow to determine an estimate for the error. Moreover, infinite repetition of the procedure should approximate the exact result to arbitrary accuracy. In other words, we want the method to be well defined in *algorithmic* form. Consequently, let us define an algorithm as *a sequence of logical and arithmetic operations (addition, subtraction, multiplication or division) which allows to approximate the solution of the problem under consideration within any accuracy desired.* This implies, of course, that numerical errors will be unavoidable.

Let us classify the occurring errors based on the structure every numerical routine follows: We have input-errors, algorithmic-errors, and output-errors as indicated schematically in Fig. 1.2. This structural classification can be refined: input-errors

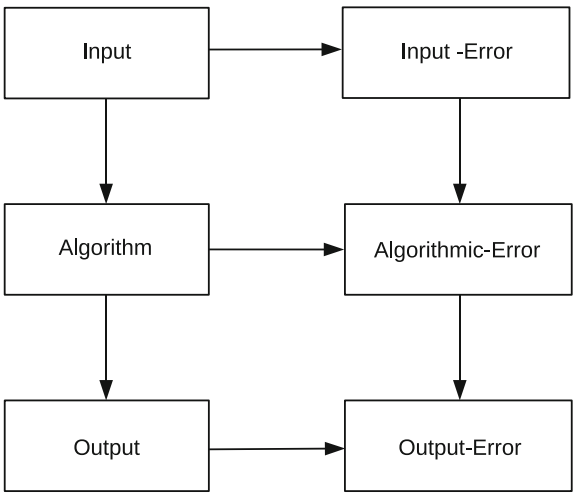


Fig. 1.2 Schematic classification of the errors occurring within a numerical procedure

are divided into rounding errors and measurement errors contained in the input data; algorithmic-errors consist of rounding errors during evaluation and of methodological errors due to mathematical approximations; finally, output errors are, in fact, rounding errors. In Sects. 1.2 and 1.3 we will concentrate on rounding errors and methodological errors. Since in most cases measurement errors cannot be influenced by the theoretical physicist concerned with numerical modeling, this particular part will not be discussed in this book. However, we will discuss the stability of numerical routines, i.e. the influence of slight modifications of the input parameters on the outcome of a particular algorithm in Sect. 1.4.

1.2 Rounding Errors

In fact, since every number is stored in a computer using a finite number of digits, we have to truncate every non-terminating number at some point. For instance, consider $\frac{2}{3} = 0.666666666666\ldots$ which will be stored as 0.6666666667 if the machine allows only ten digits. Actually, computers use binary arithmetic (for which even $0.1_{10} = 0.000110011001100\ldots_2$ is problematic) but for the moment we shall ignore this fact since the above example suffices to illustrate the crucial point. Let $\mathrm{Fl}(x)$ denote the floating-point form of a number x within the numerical range of the machine. For the above example, i.e. a ten digit storage, we have

$$\mathrm{Fl}\left(\frac{2}{3}\right) = 0.6666666667. \tag{1.20}$$

This has the consequence that, for instance, $\mathrm{Fl}(\sqrt{3}) \cdot \mathrm{Fl}(\sqrt{3}) \neq \mathrm{Fl}(\sqrt{3} \cdot \sqrt{3}) = 3$. However, $\mathrm{Fl}(\sqrt{3}) \cdot \mathrm{Fl}(\sqrt{3}) \approx 3$ within the defined range. Before we continue our discussion on rounding errors we have to introduce the concepts of the absolute and the relative error. We denote the true value of a quantity by y and its approximate value by $\overline{y}$. Then the absolute error ε_a is defined as

$$\varepsilon_a = |y - \overline{y}|, \tag{1.21}$$

while the relative error ε_r is given by

$$\varepsilon_r = \left|\frac{y - \overline{y}}{y}\right| = \frac{\varepsilon_a}{|y|}, \tag{1.22}$$

provided that $y \neq 0$. In most applications, the relative error is more significant. This is illustrated in Table 1.1, where it is intuitively obvious that in the second case the approximate value is much better although the absolute error is the same for both examples.

Table 1.1 Illustration of the significance of the relative error

	y	$\overline{y}$	ε_a	ε_r
(1)	0.1	0.09	0.01	0.1
(2)	1000.0	999.99	0.01	0.00001

Let us have a look at the relative error of an arbitrary number stored to the k-th digit: We can write an arbitrary number y in the form $y = 0.d_1d_2d_3 \dots d_kd_{k+1} \dots 10^n$ with $d_1 \neq 0$ and $n \in \mathbb{Z}$. Accordingly, we write its approximate value as $\overline{y} = 0.d_1d_2d_3 \dots d_k10^n$, where k is the maximum number of digits stored by the machine. Hence we obtain for the relative error

$$\begin{aligned}
\varepsilon_r &= \left| \frac{0.d_1d_2d_3 \dots d_kd_{k+1} \dots 10^n - 0.d_1d_2d_3 \dots d_k10^n}{0.d_1d_2d_3 \dots d_kd_{k+1} \dots 10^n} \right| \\
&= \left| \frac{0.d_{k+1}d_{k+2} \dots 10^{n-k}}{0.d_1d_2d_3 \dots 10^n} \right| \\
&= \left| \frac{0.d_{k+1}d_{k+2} \dots}{0.d_1d_2d_3 \dots} \right| 10^{-k} \\
&\leq \frac{1}{0.1} 10^{-k} \\
&= 10^{-k+1}.
\end{aligned} \tag{1.23}$$

In the last steps we employed that, since $d_1 \neq 0$, we have $0.d_1d_2d_3 \cdots \geq 0.1$ and accordingly $0.d_{k+1}d_{k+2} \cdots < 1$. If the last digit would have been rounded (for $d_{k+1} \geq 5$ we set $d_k = d_k + 1$ otherwise d_k remains unchanged) instead of a simple truncation, the relative error of a variable y would be $\varepsilon_r = 0.5 \cdot 10^{-k+1}$.

Whenever an arithmetic operation is performed, the errors of the variables involved is transferred to the result. This can occur in an advantageous or disadvantageous way, where we understand disadvantageous as an increase in the relative error. Particular care is required when two nearly identical numbers are subtracted (*subtractive cancellation*) or when a large number is divided by a, in comparison, small number. In such cases the rounding error will increase dramatically. We note that it might be necessary to avoid such operations in our aim to design an algorithm which is required to produce reasonable results. An illustrative example and its remedy will be discussed in Sect. 1.3. However, before proceeding to the next section we introduce a lower bound to the accuracy which is achievable with a non-ideal computer, the *machine-number*. The machine-number is smallest positive number η which can be added to another number, such that a change in the result is observed. In particular,

$$\eta = \min_{\delta} \left\{ \delta > 0 \middle| 1 + \delta > 1 \right\}. \tag{1.24}$$

For a (nonexistent) super-computer, which is capable of saving as much digits as desired, η would be arbitrarily small. A typical value for double-precision in FORTRAN or C is $\eta \approx 10^{-16}$.

1.3 Methodological Errors

A methodological error is introduced into the routine whenever a complex mathematical expression is replaced by an approximate, simpler one. We already came across an example when we regarded the series representation of the elliptic integral (1.14) in Sect. 1.1. Although we could evaluate the series up to an arbitrary order N, we are definitely not able to sum up the coefficients to order ∞. Hence, it is not possible to get rid of methodological errors whenever we have to deal with expressions we cannot evaluate analytically. Another intriguing example is the numerical differentiation of a (analytically) given function. The standard approximation of a derivative reads

$$f'(x_0) = \frac{\mathrm{d}}{\mathrm{d}x} f(x)\bigg|_{x=x_0} \approx \frac{f(x_0+h) - f(x_0)}{h}. \tag{1.25}$$

This approximation is referred to as *finite difference* and will be discussed in more detail in Chap. 2. One would, in a first guess, expect that the obtained value gets closer to the true value of the derivative $f'(x_0)$ with decreasing values of h. From a calculus point of view, this is correct since by definition

$$\frac{\mathrm{d}}{\mathrm{d}x} f(x)\bigg|_{x=x_0} = \lim_{h\to 0} \frac{f(x_0+h) - f(x_0)}{h}. \tag{1.26}$$

However, this is not the case numerically. In particular, one can find a value $\hat{h}$ for which the relative error is minimal, while for values $h < \hat{h}$ and $h > \hat{h}$ the approximation obtained is worse in comparison. The reason is that for small values of h the rounding errors dominate the result since $f(x_0 + h)$ and $f(x_0)$ almost cancel while $1/h$ is very small. For $h > \hat{h}$, the methodological error, i.e. the replacement of a derivative by a finite difference, controls the result.

We give one further example [6] in order to illustrate the interplay between methodological errors and rounding errors. We regard the, apparently nonhazardous, numerical solution of a quadratic equation

$$ax^2 + bx + c = 0, \tag{1.27}$$

where $a, b, c \in \mathbb{R}$, $a \neq 0$. The well known solutions read

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{and} \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}. \tag{1.28}$$

Cautious because of the above examples, we immediately diagnose the danger of a subtractive cancellation in the expression of x_1 for $b > 0$ or in x_2 for $b < 0$, and rewrite the above expression for x_1:

$$x_1 = \frac{(-b + \sqrt{b^2 - 4ac})}{2a} \frac{(-b - \sqrt{b^2 - 4ac})}{(-b - \sqrt{b^2 - 4ac})} = \frac{2c}{-b - \sqrt{b^2 - 4ac}}. \tag{1.29}$$

For x_2 we obtain

$$x_2 = \frac{2c}{-b + \sqrt{b^2 - 4ac}}. \tag{1.30}$$

Consequently, if $b > 0$ x_1 should be calculated using Eq. (1.29) and if $b < 0$ Eq. (1.28) should be used to calculate x_2. Moreover, the above expressions can be cast into one expression by setting

$$x_1 = \frac{q}{a} \qquad \text{and} \qquad x_2 = \frac{c}{q}, \tag{1.31}$$

with

$$q = -\frac{1}{2}\left[b + \text{sgn}(b)\sqrt{b^2 - 4ac}\right]. \tag{1.32}$$

Thus, Eqs. (1.31) and (1.32) can be used to calculate x_1 and x_2 for any sign of b.

1.4 Stability

When a new numerical method is designed *stability* is the third crucial point after rounding errors and methodological errors. We give an introductory definition:

An algorithm, equation or, even more general, a problem is referred to as unstable or ill-conditioned if small changes in the input cause a large change in the output.

It will be followed by a couple of elucidating examples [1].[1] To be more specific, let us now, for instance, consider the following system of equations

$$\begin{aligned} x + y &= 2.0, \\ x + 1.01y &= 2.01. \end{aligned} \tag{1.33}$$

The equations are easily solved and give $x = 1.0$ and $y = 1.0$. To make our point we consider now the case in which the right hand side of the second equation of (1.33) is subjected to a small perturbation, i.e. we consider in particular the following system of equations

[1] Although unstable behavior is not desirable in the first place the discovery of unstable systems was the birth of a specific branch in physics called *Chaos Theory*. We will come back to this point at the end of this section.

$$\begin{aligned} x + y &= 2.0, \\ x + 1.01y &= 2.02. \end{aligned} \tag{1.34}$$

The corresponding solution is $x = 0.0$ and $y = 2.0$. We observe that a relative change of 0.05 % on the right hand side of the second equation in (1.33) resulted in a 100 % relative change of the solution. Moreover, if the coefficient of y in the second equation of (1.33) were 1.0 instead of 1.01, which corresponds to a relative change of 1 %, the equations would be unsolvable. This is a behavior typical for ill-conditioned problems which, for obvious reasons, should be avoided whenever possible.

We give a second example: We consider the following initial value problem

$$\begin{cases} \ddot{y} - 10\dot{y} - 11y = 0, \\ y(0) = 1, \quad \dot{y}(0) = -1. \end{cases} \tag{1.35}$$

The general solution is readily obtained to be of the form

$$y = A \exp(-x) + B \exp(11x), \tag{1.36}$$

with numerical constants A and B. The initial conditions yield the unique solution

$$y = \exp(-x). \tag{1.37}$$

The initial conditions are now changed by two small parameters $\delta, \varepsilon > 0$ to give:

$$y(0) = 1 + \delta \qquad \text{and} \qquad \dot{y}(0) = -1 + \varepsilon. \tag{1.38}$$

The unique solution which satisfies these initial conditions is:

$$\overline{y} = \left(1 + \frac{11\delta}{12} - \frac{\varepsilon}{12}\right) \exp(-x) + \left(\frac{\delta}{12} + \frac{\varepsilon}{12}\right) \exp(11x). \tag{1.39}$$

We calculate the relative error

$$\begin{aligned} \varepsilon_r &= \left| \frac{y - \overline{y}}{y} \right| \\ &= \left(\frac{11\delta}{12} - \frac{\varepsilon}{12}\right) + \left(\frac{\delta}{12} + \frac{\varepsilon}{12}\right) \exp(12x), \end{aligned} \tag{1.40}$$

which indicates that the problem is ill-conditioned since for large values of x the second term definitely overrules the first one.

Another, but not less serious kind of problem is *induced instability*:

A method is referred to as induced unstable if a small error at one point of the calculation induces a large error at some subsequent point.

Induced instability is particularly dangerous since small rounding errors are unavoidable in most calculations. Hence, if some part of the whole algorithm is ill-conditioned, the final output will be dominated by the error induced in such a way. Again, an example will help to illustrate such behavior. The definite integral

$$I_n = \int_0^1 \mathrm{d}x\, x^n \exp(x-1), \tag{1.41}$$

is considered. Integration by parts yields

$$I_n = 1 - nI_{n-1}. \tag{1.42}$$

This expression can be used to recursively calculate I_n from I_0, where

$$I_0 = 1 - \exp(-1). \tag{1.43}$$

Although the recursion formula (1.42) is exact we will run into massive problems using it. The reason is easily illustrated:

$$\begin{aligned} I_n &= 1 - nI_{n-1} \\ &= 1 - n + n(n-1)I_{n-2} \\ &= 1 - n + n(n-1) - n(n-1)(n-2)I_{n-3} \\ &\;\;\vdots \\ &= 1 + \sum_{k=1}^{n-1}(-1)^k \frac{n!}{(n-k)!} + (-1)^{n-1} n! I_0. \end{aligned} \tag{1.44}$$

Thus, the initial rounding error included in the numerical value of I_0 is multiplied with $n!$. Note that for large n we have according to STIRLING's approximation

$$n! \approx \sqrt{2\pi} n^{n+\frac{1}{2}} \exp(-n), \tag{1.45}$$

i.e. an initial error increases almost as n^n.

However, Equation (1.42) can be reformulated to give

$$I_n = \frac{1}{n+1}(1 - I_{n+1}), \tag{1.46}$$

and this opens an alternative method for a recursive calculation of I_n. We can start with some value $N \gg n$ and simply set $I_N = 0$. The error introduced in such a way may in the end not be acceptable, nevertheless, it decreases with every iteration step due to the division by n in Eq. (1.46).

Having discussed some basic features of stability in numerical algorithms we would like to add a few remarks on *Chaos Theory*. Chaos theory investigates dynamical processes which are very sensitive to initial conditions. One of the best known examples for such a behavior is the weather prediction. Although, POINCARÉ already observed chaotic behavior while working on the three body problem, one of the pioneers of chaos theory was E. N. LORENZ [7] (not to be confused with H. LORENTZ, who introduced the LORENTZ transformation). In 1961 he ran weather simulations on a small computer. However, when he tried to reproduce one particular result by restarting the calculation with new parameters calculated the days before, he observed that the outcome was completely different. The reason was that the equations he dealt with were ill-conditioned, and the rounding error he introduced by simply typing in the numbers of the graphical output, increased drastically, and, hence, produced a completely different result. Nowadays, various physical systems are known which indeed behave in such a way. Further examples are turbulences in fluids, oscillations in electrical circuits, oscillating chemical reactions, population growth in ecology, the time evolution of the magnetic field of celestial bodies, ….

It is important to note, that chaotic behavior induced in such systems is *deterministic*, yet *unpredictable*. This is due to the impossibility of an exact knowledge of the initial conditions required to predict, for instance, the weather over a reasonably long period. A feature which is referred to as the *butterfly effect*: a hurricane can form because a butterfly flapped its wings several weeks before. However, these effects have nothing to do with intrinsic in-determinism, which is solely a feature of quantum mechanics. In contrast to this, in chaos theory, the future is uniquely determined by initial conditions, however, still unpredictable. This is often referred to as *deterministic chaos*.

It has to be emphasized that chaos in physical systems is a consequence of deterministic equations describing the processes and not a consequence of the numerical method one uses for modeling. Therefore, it is important to distinguish between the stability of a numerical method and the stability of a physical system in general.

We will come across chaotic behavior again in Sect. 6.3 where we discuss chaotic behavior in the dynamics of the double pendulum.

1.5 Concluding Remarks

In this chapter we dealt with the basic features of numerical errors one is always confronted with when developing an algorithm. One point we neglected in our discussion is the *computational cost*, i.e. the time a program needs to be executed. Although this is a very important point, it is beyond the scope of this book. However, one has to find a balance between the need of achieving the most accurate result and the computing time required to achieve it. The most accurate result is useless if the programmer does not get the result within his lifetime. D. ADAMS [8] put in a nutshell: the super-computer *Deep Thought* was asked to compute the answer to *"The Ultimate Question of Life, the Universe and Everything"*, quote:

"How long?" he said.
"Seven and a half million years."

Another quite crucial point, which we neglected so far, is the error analysis of a computational method which is based on random numbers (in fact it is pseudo-random numbers, we will come back to this point in the second part of this book). In this case, the situation changes completely, because, similar to experimental results, the observed values are distributed around a mean with a certain variance. Such results have to be interpreted in a statistical context, however it turns out that for many problems the computational efficiency can be significantly increased using such methods. Typical applications are estimates of integrals or solutions to optimization problems. Such topics will be treated in the second part of this book.

References

1. Jacques, I., Judd, C.: Numerical Analysis. Chapman and Hall, London (1987)
2. Scheck, F.: Mechanics, 5th edn. Springer, Berlin (2010)
3. Abramovitz, M., Stegun, I.A. (eds.): Handbook of Mathemathical Functions. Dover, New York (1965)
4. Mathai, A.M., Haubold, H.J.: Special Functions for Applied Scientists. Springer, Berlin (2008)
5. Beals, R., Wong, R.: Special Functions. Cambridge Studies in Advanced Mathematics. Cambridge University Press, Cambridge (2010)
6. Burden, R.L., Faires, J.D.: Numerical Analysis. PWS-Kent Publishing Comp, Boston (1993)
7. Lorenz, E.N.: Deterministic nonperiodic flow. J. Atmos. Sci. **20**, 130–141 (1963)
8. Adams, D.: The Hitchhiker's Guide to the Galaxy. Pan Books, London (1979)

If $f(x)$ is a polynomial of the form Eq. (2.2), the following results, from the theory of equations would be useful in locating its roots.

(i) Every polynomial equation of the nth degree has n and only n roots.

(ii) If n is odd, the polynomial equation has atleast one real root whose sign is opposite to that of the last term.

(iii) If n is even and the constant term is negative, then the equation has atleast one positive root and atleast one negative root.

(iv) If the polynomial equation has (a) real coefficients, then imaginary roots occur in pairs and (b) rational coefficients, then irrational roots occur in pairs.

(v) *Descartes' Rule of Signs*

(a) A polynomial equation $f(x) = 0$ cannot have more number of positive real roots than the number of changes of sign in the coefficients of $f(x)$.

(b) In (a) above, $f(x) = 0$ cannot have more number of negative real roots than the number of changes of sign in the coefficients of $f(-x)$.

2.2 BISECTION METHOD

This method is based on Theorem 1.1 which states that if a function $f(x)$ is continuous between a and b, and $f(a)$ and $f(b)$ are of opposite signs, then there exists at least one root between a and b. For definiteness, let $f(a)$ be negative and $f(b)$ be positive. Then the root lies between a and b and let its approximate value be given by $x_0 = (a + b)/2$. If $f(x_0) = 0$, we conclude that x_0 is a root of the equation $f(x) = 0$. Otherwise, the root lies either between x_0 and b, or between x_0 and a depending on whether $f(x_0)$ is negative or positive. We designate this new interval as $[a_1, b_1]$ whose length is $|b - a|/2$. As before, this is bisected at x_1 and the new interval will be exactly half the length of the previous one. We repeat this process until the latest interval (which contains the root) is as small as desired, say ε. It is clear that the interval width is reduced by a factor of one-half at each step and at the end of the nth step, the new interval will be $[a_n, b_n]$ of length $|b - a|/2^n$. We then have

$$\frac{|b-a|}{2^n} \leq \varepsilon,$$

which gives on simplification

$$n \geq \frac{\log_e(|b-a|/\varepsilon)}{\log_e 2} \tag{2.3}$$

Equation (2.3) gives the number of iterations required to achieve an accuracy ε. For example, if $|b - a| = 1$ and $\varepsilon = 0.001$, then it can be seen that

$$n \geq 10 \tag{2.4}$$

The method is shown graphically in Fig. 2.1.

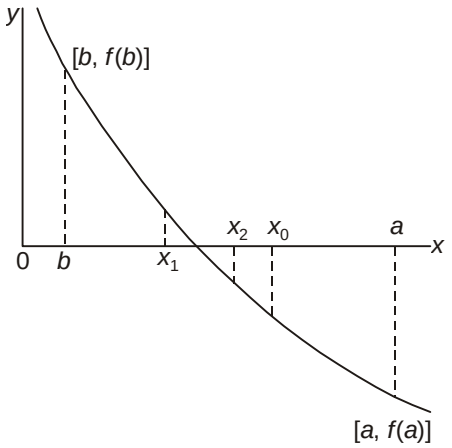


Figure 2.1 Graphical representation of the bisection method.

It should be noted that this method always succeeds. If there are more roots than one in the interval, bisection method finds one of the roots. It can be easily programmed using the following computational steps:

1. Choose two real numbers a and b such that $f(a)\, f(b) < 0$.
2. Set $x_r = (a + b)/2$.
3. (a) If $f(a)\, f(x_r) < 0$, the root lies in the interval (a, x_r). Then, set $b = x_r$ and go to step 2 above.
   (b) If $f(a)\, f(x_r) > 0$, the root lies in the interval (x_r, b). Then, set $a = x_r$ and go to step 2.
   (c) If $f(a)\, f(x_r) = 0$, it means that x_r is a root of the equation $f(x) = 0$ and the computation may be terminated.

In practical problems, the roots may not be exact so that condition (c) above is never satisfied. In such a case, we need to adopt a criterion for deciding when to terminate the computations.

A convenient criterion is to compute the percentage error ε_r defined by

$$\varepsilon_r = \left| \frac{x_r' - x_r}{x_r'} \right| \times 100\%. \tag{2.5}$$

where x_r' is the new value of x_r. The computations can be terminated when ε_r becomes less than a prescribed tolerance, say ε_p. In addition, the maximum number of iterations may also be specified in advance.

Example 2.1 Find a real root of the equation $f(x) = x^3 - x - 1 = 0$.

Since $f(1)$ is negative and $f(2)$ positive, a root lies between 1 and 2 and, therefore, we take $x_0 = 3/2$. Then

$$f(x_0) = \frac{27}{8} - \frac{3}{2} = \frac{15}{8}, \text{ which is positive.}$$

Hence the root lies between 1 and 1.5 and we obtain

$$x_1 = \frac{1+1.5}{2} = 1.25$$

We find $f(x_1) = -19/64$, which is negative. We, therefore, conclude that the root lies between 1.25 and 1.5. If follows that

$$x_2 = \frac{1.25+1.5}{2} = 1.375$$

The procedure is repeated and the successive approximations are

$$x_3 = 1.3125, \quad x_4 = 1.34375, \quad x_5 = 1.328125, \text{ etc.}$$

Example 2.2 Find a real root of the equation $x^3 - 2x - 5 = 0$.

Let $f(x) = x^3 - 2x - 5$. Then

$$f(2) = -1 \quad \text{and} \quad f(3) = 16.$$

Hence a root lies between 2 and 3 and we take

$$x_1 = \frac{2+3}{2} = 2.5$$

Since $f(x_1) = f(2.5) = 5.6250$, the root lies between 2 and 2.25.
Hence

$$x_2 = \frac{2+2.5}{2} = 2.25$$

Now, $f(x_2) = 1.890625$, the root lies between 2 and 2.25.
Therefore,

$$x_3 = \frac{2+2.25}{2} = 2.125$$

Since $f(x_3) = 0.3457$, the root lies between 2 and 2.125.
Therefore,

$$x_4 = \frac{2+2.125}{2} = 2.0625$$

Proceeding in this way, we obtain the successive approximations:

$$x_5 = 2.09375, \quad x_6 = 2.10938, \quad x_7 = 2.10156,$$
$$x_8 = 2.09766, \quad x_9 = 2.09570, \quad x_{10} = 2.09473,$$
$$x_{11} = 2.09424, \ldots$$

We find

$$x_{11} - x_{10} = -0.0005,$$

and

$$\left|\frac{x_{11} - x_{10}}{x_{11}}\right| \times 100 = \frac{0.0005}{2.09424} \times 100 = 0.02\%$$

Hence a root, correct to three decimal places, is 2.094.

Example 2.3 Find a real root of $f(x) = x^3 + x^2 + x + 7 = 0$ correct to three decimal places.

The given equation is a cubic and the last term is positive. Hence, $f(x) = 0$ will have a negative real root. We find that

$$f(-1) = 6, \quad f(-2) = 1 \text{ and } f(-3) = -14.$$

Therefore, a real root lies between –3 and –2.

We take

$$x_1 = \frac{-2-3}{2} = -2.5$$

Since $f(-2.5) = -4.875$, the root lies between –2 and –2.5, and then

$$x_2 = \frac{-2-2.5}{2} = -2.25$$

Now $f(x_2) = -1.5781$, and, therefore, the root lies between –2 and –2.25.

It follows that

$$x_3 = \frac{-4.25}{2} = -2.125$$

Successive approximations are given by

$$x_4 = -2.0625, \quad x_5 = -2.0938, \quad x_6 = -2.1094,$$
$$x_7 = -2.1016, \quad x_8 = -2.1055, \quad x_9 = -2.1035,$$
$$x_{10} = -2.1045, \quad x_{11} = -2.1050, \ldots$$

The difference between x_{10} and x_{11} is 0.0005. Hence, we conclude that the root is given by $x = -2.105$, correct to three decimal places.

Example 2.4 Find the positive root, between 0 and 1, of the equation $x = e^{-x}$ to a tolerance of 0.05%.

Let

$$f(x) = xe^x - 1 = 0$$

We have, $f(0) = -1$ and $f(1) = e - 1$, which is positive. Hence, a root exists between 0 and 1, and

$$x_1 = \frac{0+1}{2} = 0.5$$

Because, $f(x_1) = -0.1756$, the root lies between 0.5 and 1.0.

Then

$$x_2 = \frac{0.5+1.0}{2} = 0.75$$

Now, the tolerance ε_1 is given by

$$\varepsilon_1 = \left|\frac{x_2 - x_1}{x_2}\right| \times 100$$

$$= \frac{0.25}{0.75} \times 100 = 33.33\%$$

since $f(x_2) = 0.5878$, the root lies between 0.5 and 0.75.
Therefore,

$$x_3 = \frac{0.5 + 0.75}{2} = 0.625$$

also,

$$\varepsilon_2 = \left|\frac{0.625 - 0.75}{0.625}\right| \times 100 = 20\%.$$

Proceeding in this way, successive approximations and tolerances are obtained:

$x_4 = 0.5625$, $\varepsilon_3 = 11.11\%$; $x_5 = 0.5938$, $\varepsilon_4 = 5.26\%$;
$x_6 = 0.5781$, $\varepsilon_5 = 2.71\%$; $x_7 = 0.5703$, $\varepsilon_6 = 1.37\%$;
$x_8 = 0.5664$, $\varepsilon_7 = 0.69\%$; $x_9 = 0.5684$, $\varepsilon_8 = 0.35\%$;
$x_{10} = 0.5674$, $\varepsilon_9 = 0.18\%$; $x_{11} = 0.5669$, $\varepsilon_{10} = 0.09\%$;
$x_{12} = 0.5671$, $\varepsilon_{11} = 0.035\%$

Since $\varepsilon_{11} = 0.035\% < 0.05\%$, the required root is 0.567, correct to three decimal places.

Example 2.5 Find a root, correct to three decimal places and lying between 0 and 0.5, of the equation

$$4e^{-x} \sin x - 1 = 0$$

Let

$$f(x) = 4e^{-x} \sin x - 1$$

We have $f(0) = -1$ and $f(0.5) = 0.163145$
Therefore,

$$x_1 = 0.25$$

Since $f(0.25) = -0.22929$, it follows that the root lies between 0.25 and 0.5.
Therefore,

$$x_2 = \frac{0.75}{2} = 0.375$$

The successive approximations are given by

$x_3 = 0.3125$, $x_4 = 0.3438$, $x_5 = 0.3594$,
$x_6 = 0.3672$, $x_7 = 0.3711$, $x_8 = 0.3692$,
$x_9 = 0.3702$, $x_{10} = 0.3706$, $x_{11} = 0.3704$
$x_{12} = 0.3705$, …

Hence the required root is 0.371, correct to three decimal places.

Hence Eq. (2.22) can be written in the simpler form

$$\xi = x_{i+1} - \frac{(\Delta x_i)^2}{\Delta^2 x_{i-1}}. \tag{2.23}$$

which explains the term Δ^2-process.

In any numerical application, the values of the following underlined quantities must be obtained.

x_{i-1}		
	Δx_{i-1}	
x_i		$\underline{\Delta^2 x_{i-1}}$
	$\underline{\Delta x_i}$	
$\underline{x_{i+1}}$		

Example 2.14 We consider again Example 2.11, viz., the equation

$$x = \frac{1}{2}(3 + \cos x)$$

As before,

$x_1 = 1.5$		
	0.035	
$x_2 = 1.535$		-0.052
	-0.017	
$x_3 = 1.518$		

Hence we obtain from Eq. (2.23)

$$x_4 = 1.518 - \frac{(-0.017)^2}{-0.052} = 1.524,$$

which corresponds to six normal iterations.

2.5 NEWTON–RAPHSON METHOD

This method is generally used to improve the result obtained by one of the previous methods. Let x_0 be an approximate root of $f(x) = 0$ and let $x_1 = x_0 + h$ be the correct root so that $f(x_1) = 0$. Expanding $f(x_0 + h)$ by Taylor's series, we obtain

$$f(x_0) + hf'(x_0) + \frac{h^2}{2!} f''(x_0) + \cdots = 0.$$

Neglecting the second and higher-order derivatives, we have

$$f(x_0) + hf'(x_0) = 0,$$

which gives

$$h = -\frac{f(x_0)}{f'(x_0)}.$$

A better approximation than x_0 is, therefore, given by x_1, where

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}. \tag{2.24a}$$

Successive approximations are given by $x_2, x_3, \ldots, x_{n+1}$, where

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \tag{2.24b}$$

which is the *Newton–Raphson formula*.

If we compare Eq. (2.24b) with the relation

$$x_{n+1} = \phi(x_n)$$

of the iterative method [see Eq. (2.9)], we obtain

$$\phi(x) = x - \frac{f(x)}{f'(x)},$$

which gives

$$\phi'(x) = \frac{f(x)\, f''(x)}{[f'(x)]^2}. \tag{2.25}$$

To examine the convergence we assume that $f(x)$, $f'(x)$ and $f''(x)$ are continuous and bounded on any interval containing the root $x = \xi$ of the equation $f(x) = 0$. If ξ is a simple root, then $f'(x) \neq 0$. Further since $f'(x)$ is continuous, $|f'(x)| \geq \varepsilon$ for some $\varepsilon > 0$ in a suitable neighbourhood of ξ. Within this neighbourhood we can select an interval such that $|f(x)\, f''(x)| < \varepsilon^2$ and this is possible since $f(\xi) = 0$ and since $f(x)$ is continuously twice differentiable. Hence, in this interval we have

$$|\phi'(x)| < 1. \tag{2.26}$$

Therefore, Newton–Raphson formula given in Eq. (2.24b) converges, provided that the initial approximation x_0 is chosen sufficiently close to ξ. When ξ is a multiple root, the Newton–Raphson method still converges but slowly. Convergence can, however, be made faster by modifying Eq. (2.24b). This will be discussed later.

To obtain the rate of convergence of the method, we note that $f(\xi) = 0$ so that Taylor's expansion gives

$$f(x_n) + (\xi - x_n) f'(x_n) + \frac{1}{2}(\xi - x_n)^2 f''(x_n) + \cdots = 0,$$

from which we obtain

$$-\frac{f(x_n)}{f'(x_n)} = (\xi - x_n) + \frac{1}{2}(\xi - x_n)^2 \frac{f''(x_n)}{f'(x_n)} \tag{2.27}$$

From Eqs. (2.24b) and (2.27), we have

$$x_{n+1} - \xi = \frac{1}{2}(x_n - \xi)^2 \frac{f''(x_n)}{f'(x_n)} \tag{2.28}$$

Setting

$$\varepsilon_n = x_n - \xi, \tag{2.29}$$

Equation (2.28) gives

$$\varepsilon_{n+1} \approx \frac{1}{2}\varepsilon_n^2 \frac{f''(\xi)}{f'(\xi)}, \tag{2.30}$$

so that the Newton–Raphson process has a second-order or quadratic convergence.

Geometrically, the method consists in replacing the part of the curve between the point $[x_0, f(x_0)]$ and the x-axis by means of the tangent to the curve at the point, and is described graphically in Fig. 2.3. It can be used for solving both algebraic and transcendental equations and it can also be used when the roots are complex.

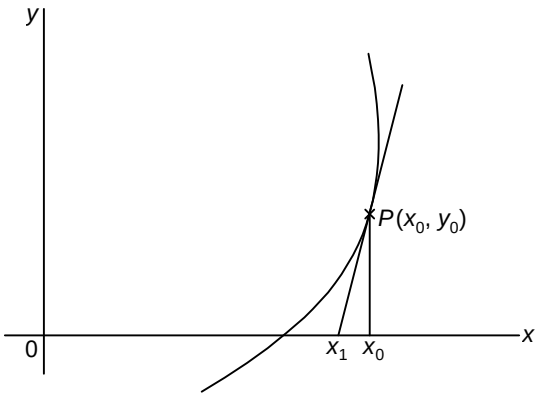


Figure 2.3 Newton–Raphson method.

Example 2.15 Use the Newton–Raphson method to find a root of the equation $x^3 - 2x - 5 = 0$.

Here $f(x) = x^3 - 2x - 5$ and $f'(x) = 3x^2 - 2$. Hence Eq. (2.24b) gives:

$$x_{n+1} = x_n - \frac{x_n^3 - 2x_n - 5}{3x_n^2 - 2} \tag{i}$$

Choosing $x_0 = 2$, we obtain $f(x_0) = -1$ and $f'(x_0) = 10$. Putting $n = 0$ in Eq. (i), we obtain

$$x_1 = 2 - \left(-\frac{1}{10}\right) = 2.1$$

Now,

$$f(x_1) = (2.1)^3 - 2(2.1) - 5 = 0.061,$$

and

$$f'(x_1) = 3(2.1)^2 - 2 = 11.23.$$

Hence

$$x_2 = 2.1 - \frac{0.061}{11.23} = 2.094568.$$

This example demonstrates that Newton–Raphson method converges more rapidly than the methods described in the previous sections, since this requires fewer iterations to obtain a specified accuracy. But since two function evaluations are required for each iteration, Newton–Raphson method requires more computing time.

Example 2.16 Find a root of the equation $x \sin x + \cos x = 0$.

We have

$$f(x) = x \sin x + \cos x \quad \text{and} \quad f'(x) = x \cos x.$$

The iteration formula is, therefore,

$$x_{n+1} = x_n - \frac{x_n \sin x_n + \cos x_n}{x_n \cos x_n}.$$

With $x_0 = \pi$, the successive iterates are given below

n	x_n	$f(x_n)$	x_{n+1}
0	3.1416	–1.0	2.8233
1	2.8233	–0.0662	2.7986
2	2.7986	–0.0006	2.7984
3	2.7984	0.0	2.7984

Example 2.17 Find a real root of the equation $x = e^{-x}$, using the Newton–Raphson method.

We write the equation in the form

$$f(x) = xe^x - 1 = 0 \tag{i}$$

Let $x_0 = 1$. Then

$$x_1 = 1 - \frac{e-1}{2e} = \frac{1}{2}\left(1 + \frac{1}{e}\right) = 0.6839397$$

Now

$$f(x_1) = 0.3553424, \quad \text{and} \quad f'(x_1) = 3.337012,$$

so that

$$x_2 = 0.6839397 - \frac{0.3553424}{3.337012} = 0.5774545$$

Proceeding in this way, we obtain

$$x_3 = 0.5672297 \quad \text{and} \quad x_4 = 0.5671433.$$

Example 2.18 Using Newton–Raphson method, find a real root, correct to 3 decimal places, of the equation $\sin x = x/2$ given that the root lies between $\pi/2$ and π.

Let

$$f(x) = \sin x - \frac{x}{2}$$

Then

$$f'(x) = \cos x - \frac{1}{2}$$

Choosing $x_0 = \frac{\pi}{2}$, we obtain

$$x_1 = \frac{\pi}{2} - \frac{\sin\frac{\pi}{2} - \frac{\pi}{4}}{-\frac{1}{2}} = 2,$$

$$x_2 = 2 - \frac{\sin 2 - 1}{\cos 2 - \frac{1}{2}} = 1.9010,$$

$$x_3 = 1.9010 - \frac{\sin 1.9010 - 0.9505}{\cos 1.9010 - 0.5} = 1.8955$$

Similarly, $x_4 = 1.8954$, $x_5 = 1.8955$, … . Hence the required root is $x = 1.896$.

Example 2.19 Given the equation $4e^{-x} \sin x - 1 = 0$, find the root between 0 and 0.5 correct to three decimal places.

Let $f(x) = 4e^{-x} \sin x - 1$ and $x_0 = 0.2$.

Then

$$f(x_0) = -0.349373236,$$

and

$$f'(x_0) = 2.559015826.$$

Therefore,

$$x_1 = 0.2 + \frac{0.349373236}{2.559015826}$$
$$= 0.336526406 = 0.33653.$$

Now,

$$f(x_1) = -0.056587$$

and

$$f'(x_1) = 1.753305735$$
$$= 1.75330$$

Therefore,

$$x_2 = 0.33653 + \frac{0.056587}{1.75330} = 0.36880$$

For the next approximation, we find $f(x_2) = -0.00277514755$ and $f'(x_2) = 1.583028705$. This gives

$$x_3 = 0.36880 - 0.00175 = 0.37055$$

since $f(x_3) = -0.00001274$, it follows that the required root is given by $x = 0.370$.

Generalized Newton's method

If ξ is a root of $f(x) = 0$ with multiplicity p, then the iteration formula corresponding to Eq. (2.24) is taken as

$$x_{n+1} = x_n - p\frac{f(x_n)}{f'(x_n)}, \tag{2.31}$$

which means that $(1/p)f'(x_n)$ is the slope of the straight line passing through (x_n, y_n) and intersecting the x-axis at the point $(x_{n+1}, 0)$.

Equation (2.31) is called the *generalized Newton's formula* and reduces to Eq. (2.24) for $p = 1$. Since ξ is a root of $f(x) = 0$ with multiplicity p, it follows that ξ is also a root of $f'(x) = 0$ with multiplicity $(p - 1)$, of $f''(x) = 0$ with multiplicity $(p - 2)$, and so on. Hence the expressions

$$x_0 - p\frac{f(x_0)}{f'(x_0)}, \quad x_0 - (p-1)\frac{f'(x_0)}{f''(x_0)}, \quad x_0 - (p-2)\frac{f''(x_0)}{f'''(x_0)}$$

must have the same value if there is a root with multiplicity p, provided that the initial approximation x_0 is chosen sufficiently close to the root.

Example 2.20 Find a double root of the equation $f(x) = x^3 - x^2 - x + 1 = 0$.

Choosing $x_0 = 0.8$, we have

$$f'(x) = 3x^3 - 2x - 1, \quad \text{and} \quad f''(x) = 6x - 2.$$

With $x_0 = 0.8$, we obtain

$$x_0 - 2\frac{f(x_0)}{f'(x_0)} = 0.8 - 2\frac{0.072}{-(0.68)} = 1.012,$$

and

$$x_0 - \frac{f'(x_0)}{f''(x_0)} = 0.8 - \frac{(-0.68)}{2.8} = 1.043.$$

3 Chapter

Interpolation

3.1 INTRODUCTION

The statement

$$y = f(x), \qquad x_0 \le x \le x_n$$

means: corresponding to every value of x in the range $x_0 \le x \le x_n$, there exists one or more values of y. Assuming that $f(x)$ is single-valued and continuous and that it is known explicitly, then the values of $f(x)$ corresponding to certain given values of x, say $x_0, x_1, \ldots, x_n$ can easily be computed and tabulated. The central problem of numerical analysis is the converse one: Given the set of tabular values $(x_0, y_0), (x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ satisfying the relation $y = f(x)$ where the explicit nature of $f(x)$ is not known, it is required to find a simpler function, say $\phi(x)$, such that $f(x)$ and $\phi(x)$ *agree at the set of tabulated points*. Such a process is called *interpolation*. If $\phi(x)$ is a polynomial, then the process is called *polynomial interpolation* and $\phi(x)$ is called the *interpolating polynomial*. Similarly, different types of interpolation arise depending on whether $\phi(x)$ is a finite trigonometric series, series of Bessel functions, etc. In this chapter, we shall be concerned with polynomial interpolation only. As a justification for the approximation of an unknown function by means of a polynomial, we state here, without proof, a famous theorem due to Weierstrass (1885): if $f(x)$ is continuous in $x_0 \le x \le x_n$, then given any $\varepsilon > 0$, there exists a polynomial $P(x)$ such that

$$|f(x) - P(x)| < \varepsilon, \quad \text{for all } x \text{ in } (x_0, x_n).$$

This means that it is possible to find a polynomial $P(x)$ whose graph remains within the region bounded by $y = f(x) - \varepsilon$ and $y = f(x) + \varepsilon$ for all x between x_0 and x_n, however small ε may be.

When approximating a given function $f(x)$ by means of polynomial $\phi(x)$, one may be tempted to ask: (i) How should the closeness of the approximation be measured? and (ii) What is the criterion to decide the best polynomial approximation to the function? Answers to these questions, important though they are for the practical problem of interpolation, are outside the scope of this book and will not be attempted here. We will, however, derive in the next section a formula for finding the error associated with the approximation of a tabulated function by means of a polynomial.

3.2 ERRORS IN POLYNOMIAL INTERPOLATION

Let the function $y(x)$, defined by the $(n + 1)$ points (x_i, y_i), $i = 0, 1, 2, \ldots, n$, be continuous and differentiable $(n + 1)$ times, and let $y(x)$ be approximated by a polynomial $\phi_n(x)$ of degree not exceeding n such that

$$\phi_n(x_i) = y_i, \quad i = 0, 1, 2, \ldots, n \tag{3.1}$$

If we now use $\phi_n(x)$ to obtain approximate values of $y(x)$ at some points other than those defined by Eq. (3.1), what would be the accuracy of this approximation? Since the expression $y(x) - \phi_n(x)$ vanishes for $x = x_0, x_1, \ldots, x_n$, we put

$$y(x) - \phi_n(x) = L\Pi_{n+1}(x), \tag{3.2}$$

where

$$\Pi_{n+1}(x) = (x - x_0)(x - x_1) \ldots (x - x_n) \tag{3.3}$$

and L is to be determined such that Eq. (3.2) holds for any intermediate value of x, say $x = x'$, $x_0 < x' < x_n$. Clearly,

$$L = \frac{y(x') - \phi_n(x')}{\Pi_{n+1}(x')}. \tag{3.4}$$

We construct a function $F(x)$ such that

$$F(x) = y(x) - \phi_n(x) - L\Pi_{n+1}(x), \tag{3.5}$$

where L is given by Eq. (3.4) above,

It is clear that

$$F(x_0) = F(x_1) = \cdots = F(x_n) = F(x') = 0,$$

that is, $F(x)$ vanishes $(n + 2)$ times in the interval $x_0 \le x \le x_n$; consequently, by the repeated application of Rolle's theorem (see Theorem 1.3, Section 1.2), $F'(x)$ must vanish $(n + 1)$ times, $F''(x)$ must vanish n times, etc., in the interval $x_0 \le x \le x_n$. In particular, $F^{(n+1)}(x)$ must vanish once in the interval.

Let this point be given by $x = \xi$, $x_0 < \xi < x_n$. On differentiating Eq. (3.5) $(n + 1)$ times with respect to x and putting $x = \xi$, we obtain

$$0 = y^{(n+1)}(\xi) - L(n + 1)!$$

so that

$$L = \frac{y^{(n+1)}(\xi)}{(n+1)!}. \tag{3.6}$$

Comparison of Eqs. (3.4) and (3.6) yields the results

$$y(x') - \phi_n(x') = \frac{y^{(n+1)}(\xi)}{(n+1)!}\Pi_{n+1}(x').$$

Dropping the prime on x', we obtain

$$y(x) - \phi_n(x) = \frac{\Pi_{n+1}(x)}{(n+1)!}y^{(n+1)}(\xi), \quad x_0 < \xi < x_n, \tag{3.7}$$

which is the required expression for the error. Since $y(x)$ is, generally, unknown and hence we do not have any information concerning $y^{(n+1)}(x)$, formula (3.7) is almost useless in practical computations. On the other hand, it is extremely useful in theoretical work in different branches of numerical analysis. In particular, we will use it to determine errors in Newton's interpolating formulae which will be discussed in Section 3.6.

3.3 FINITE DIFFERENCES

Assume that we have a table of values (x_i, y_i), $i = 0, 1, 2, \ldots, n$ of any function $y = f(x)$, the values of x being equally spaced, i.e., $x_i = x_0 + ih$, $i = 0, 1, 2, \ldots, n$. Suppose that we are required to recover the values of $f(x)$ for some intermediate values of x, or to obtain the derivative of $f(x)$ for some x in the range $x_0 \le x \le x_n$. The methods for the solution to these problems are based on the concept of the 'differences' of a function which we now proceed to define.

3.3.1 Forward Differences

If $y_0, y_1, y_2, \ldots, y_n$ denote a set of values of y, then $y_1 - y_0, y_2 - y_1, \ldots, y_n - y_{n-1}$ are called the *differences* of y. Denoting these differences by $\Delta y_0, \Delta y_1, \ldots, \Delta y_{n-1}$ respectively, we have

$$\Delta y_0 = y_1 - y_0, \qquad \Delta y_1 = y_2 - y_1, \ \ldots, \qquad \Delta y_{n-1} = y_n - y_{n-1},$$

where Δ is called the *forward difference operator* and $\Delta y_0, \Delta y_1, \ldots$, are called *first forward differences*. The differences of the first forward differences are called *second forward differences* and are denoted by $\Delta^2 y_0, \Delta^2 y_1, \ldots$ Similarly, one can define *third forward differences*, *fourth forward differences*, etc.

Thus,

$$\begin{aligned}\Delta^2 y_0 = \Delta y_1 - \Delta y_0 &= y_2 - y_1 - (y_1 - y_0)\\ &= y_2 - 2y_1 + y_0,\\ \Delta^3 y_0 = \Delta^2 y_1 - \Delta^2 y_0 &= y_3 - 2y_2 + y_1 - (y_2 - 2y_1 + y_0)\\ &= y_3 - 3y_2 + 3y_1 - y_0\\ \Delta^4 y_0 = \Delta^3 y_1 - \Delta^3 y_0 &= y_4 - 3y_3 + 3y_2 - y_1 - (y_3 - 3y_2 + 3y_1 - y_0)\\ &= y_4 - 4y_3 + 6y_2 - 4y_1 + y_0.\end{aligned}$$

It is, therefore, clear that any higher-order difference can easily be expressed in terms of the ordinates, since the coefficients occurring on the right side are the binomial coefficients.

Table 3.1 shows how the forward differences of all orders can be formed:

Table 3.1 Forward Difference Table

x	y_0	Δ	Δ^2	Δ^3	Δ^4	Δ^5	Δ^6
x_0	y_0						
		Δy_0					
x_1	y_1		$\Delta^2 y_0$				
		Δy_1		$\Delta^3 y_0$			
x_2	y_2		$\Delta^2 y_1$		$\Delta^4 y_0$		
		Δy_2		$\Delta^3 y_1$		$\Delta^5 y_0$	
x_3	y_3		$\Delta^2 y_2$		$\Delta^4 y_1$		$\Delta^6 y_0$
		Δy_3		$\Delta^3 y_2$		$\Delta^5 y_1$	
x_4	y_4		$\Delta^2 y_3$		$\Delta^4 y_2$		
		Δy_4		$\Delta^3 y_3$			
x_5	y_5		$\Delta^2 y_4$				
		Δy_5					
x_6	y_6						

In practical computations, the forward difference table can be formed in the following way. For the data points (x_i, y_i), $i = 0, 1, 2, \ldots, n$ and $x_i = x_0 + ih$, we have

$$\Delta y_j = y_{j+1} - y_i, j = 0, 1, \ldots, n - 1.$$

Denoting y_j as DEL(0, j), the above equation can be written as

$$\Delta y_j = \text{DEL}(0, j + 1) - \text{DEL}(0, j) = \text{DEL}(1, j)$$

It follows that

$$\Delta^i y_j = \text{DEL}(i - 1, j + 1) - \text{DEL}(i - 1, j),$$

which is the ith forward difference of y_j.

For the data points (x_i, y_i), $i = 0, 1, 2, \ldots, 6$, we have difference Table 3.2.

Table 3.2 Forward Difference Table

x	y	Δ	Δ^2	Δ^3	Δ^4	Δ^5	Δ^6
x_0	DEL(0, 0)						
		DEL(1, 0)					
x_1	DEL(0, 1)		DEL(2, 0)				
		DEL(1, 1)		DEL(3, 0)			
x_2	DEL(0, 2)		DEL(2, 1)		DEL(4, 0)		
		DEL(1, 2)		DEL(3, 1)		DEL(5, 0)	
x_3	DEL(0, 3)		DEL(2, 2)		DEL(4, 1)		DEL(6, 0)
		DEL(1, 3)		DEL(3, 2)		DEL(5, 1)	
x_4	DEL(0, 4)		DEL(2, 3)		DEL(4, 2)		
		DEL(1, 4)		DEL(3, 3)			
x_5	DEL(0, 5)		DEL(2, 4)				
		DEL(1, 5)					
x_6	DEL(0, 6)						

In Table 3.2

$$\begin{aligned}
\text{DEL}(4, 0) &= \text{DEL}(3, 1) - \text{DEL}(3, 0) \\
&= \text{DEL}(2, 2) - \text{DEL}(2, 1) - [\text{DEL}(2, 1) - \text{DEL}(2, 0)] \\
&= \text{DEL}(1, 3) - \text{DEL}(1, 2) - 2[\text{DEL}(1, 2) - \text{DEL}(1, 1)] \\
&\quad + \text{DEL}(1, 1) - \text{DEL}(1, 0) \\
&= \text{DEL}(0, 4) - \text{DEL}(0, 3) - 3[\text{DEL}(0, 3) - \text{DEL}(0, 2)] \\
&\quad + 3[\text{DEL}(0, 2) - \text{DEL}(0, 1)] - [\text{DEL}(0, 1) - \text{DEL}(0, 0)] \\
&= \text{DEL}(0, 4) - 4\text{DEL}(0, 3) + 6\text{DEL}(0, 2) - 4\text{DEL}(0, 1) + \text{DEL}(0, 0) \\
&= y_4 - 4y_3 + 6y_2 - 4y_1 + y_0
\end{aligned}$$

The forward difference table can now be formed by the simple statements:

```
Do i = 1(1)n
Do j = 0(1)n - i
DEL(i, j) = DEL(i - 1, j + 1) - DEL(i - 1, j)
Next j
Next i
End
```

3.3.2 Backward Differences

The differences $y_1 - y_0$, $y_2 - y_1$, …, $y_n - y_{n-1}$ are called first *backward differences* if they are denoted by ∇y_1, ∇y_2, …, ∇y_n respectively, so that

$$\begin{aligned}
\nabla y_1 &= y_1 - y_0, \quad \nabla y_2 = y_2 - y_1, \\
&\vdots \quad \vdots \quad \vdots \quad \vdots \\
\nabla y_n &= y_n - y_{n-1},
\end{aligned}$$

where ∇ is called the *backward difference operator.* In a similar way, one can define backward differences of higher orders.

Thus the nth difference is $a_0 n! h^n$, which is a constant. Hence, the $(n+1)$th, and higher differences of a polynomial of nth degree will be zero. Conversely, if the nth differences of a tabulated function are constant and the $(n+1)$th, $(n+2)$th, …, differences all vanish, then the tabulated function represents a polynomial of degree n. It should be noted that these results hold good only if the values of x are equally spaced. The converse is important in numerical analysis since it enables us to approximate a function by a polynomial if its differences of some order become nearly constant.

3.6 NEWTON'S FORMULAE FOR INTERPOLATION

Given the set of $(n+1)$ values, viz., (x_0, y_0), (x_1, y_1), (x_2, y_2), …, (x_n, y_n), of x and y, it is required to find $y_n(x)$, a polynomial of the nth degree such that y and $y_n(x)$ agree at the tabulated points. Let the values of x be equidistant, i.e. let

$$x_i = x_0 + ih, \quad i = 0, 1, 2, \ldots, n.$$

Since $y_n(x)$ is a polynomial of the nth degree, it may be written as

$$\left.\begin{aligned} y_n(x) &= a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) \\ &\quad + a_3(x - x_0)(x - x_1)(x - x_2) + \cdots \\ &\quad + a_n(x - x_0)(x - x_1)(x - x_2)\ldots(x - x_{n-1}). \end{aligned}\right\} \tag{3.9}$$

Imposing now the condition that y and $y_n(x)$ should agree at the set of tabulated points, we obtain

$$a_0 = y_0;\ a_1 = \frac{y_1 - y_0}{x_1 - x_0} = \frac{\Delta y_0}{h};\ a_2 = \frac{\Delta^2 y_0}{h^2 2!};\ a_3 = \frac{\Delta^3 y_0}{h^3 3!};\ \cdots;\ a_n = \frac{\Delta^n y_0}{h^n n!};$$

Setting $x = x_0 + ph$ and substituting for $a_0, a_1, \ldots, a_n$, Eq. (3.9) gives

$$y_n(x) = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \cdots$$

$$+ \frac{p(p-1)(p-2)\ldots(p-n+1)}{n!}\Delta^n y_0, \tag{3.10}$$

which is *Newton's forward difference interpolation formula* and is useful for interpolation *near the beginning* of a set of tabular values.

To find the error committed in replacing the function $y(x)$ by means of the polynomial $y_n(x)$, we use Eq. (3.7) to obtain

$$y(x) - y_n(x) = \frac{(x - x_0)(x - x_1)\ldots(x - x_n)}{(n+1)!} y^{(n+1)}(\xi), \qquad x_0 < \xi < x_n \tag{3.11}$$

As remarked earlier we do not have any information concerning $y^{(n+1)}(x)$, and therefore, formula given in Eq. (3.11) is useless in practice. Nevertheless,

if $y^{(n+1)}(x)$ does not vary too rapidly in the interval, a useful estimate of the derivative can be obtained in the following way. Expanding $y(x+h)$ by Taylor's series (see Theorem 1.4), we obtain

$$y(x+h) = y(x) + hy'(x) + \frac{h^2}{2!}y''(x) + \cdots$$

Neglecting the terms containing h^2 and higher powers of h, this gives

$$y'(x) \approx \frac{1}{h}[y(x+h) - y(x)] = \frac{1}{h}\Delta y(x).$$

Writing $y'(x)$ as $Dy(x)$ where $D \equiv d/dx$, the differentiation operator, the above equation gives the operator relation

$$D \equiv \frac{1}{h}\Delta \quad \text{and so} \quad D^{n+1} \equiv \frac{1}{h^{n+1}}\Delta^{n+1}.$$

We thus obtain

$$y^{(n+1)}(x) \approx \frac{1}{h^{n+1}}\Delta^{n+1}y(x). \tag{3.12}$$

Equation (3.11) can, therefore, be written as

$$y(x) - y_n(x) = \frac{p(p-1)(p-2)\ldots(p-n)}{(n+1)!}\Delta^{n+1}y(\xi) \tag{3.13}$$

in which form it is suitable for computation.

Instead of assuming $y_n(x)$ as in Eq. (3.9), if we choose it in the form

$$\begin{aligned} y_n(x) = a_0 &+ a_1(x - x_n) + a_2(x - x_n)(x - x_{n-1}) \\ &+ a_3(x - x_n)(x - x_{n-1})(x - x_{n-2}) + \cdots \\ &+ a_n(x - x_n)(x - x_{n-1})\ldots(x - x_1). \end{aligned}$$

and then impose the condition that y and $y_n(x)$ should agree at the tabulated points x_n, x_{n-1}, …, x_2, x_1, x_0, we obtain (after some simplification)

$$y_n(x) = y_n + p\nabla y_n + \frac{p(p+1)}{2!}\nabla^2 y_n + \cdots + \frac{p(p+1)\ldots(p+n-1)}{n!}\nabla^n y_n, \tag{3.14}$$

where $p = (x - x_n)/h$.

This is *Newton's backward difference interpolation formula* and it uses tabular values to the left of y_n. This formula is therefore useful for interpolation *near the end of* the tabular values.

It can be shown that the error in this formula may be written as

$$y(x) - y_n(x) = \frac{p(p+1)(p+2)\ldots(p+n)}{(n+1)!}\nabla^{n+1}y(\xi), \tag{3.15}$$

where $x_0 < x < x_n$ and $x = x_n + ph$.

The following examples illustrate the use of these formulae.

Example 3.4 Find the cubic polynomial which takes the following values: $y(1) = 24$, $y(3) = 120$, $y(5) = 336$, and $y(7) = 720$. Hence, or otherwise, obtain the value of $y(8)$.

We form the difference table:

x	y	Δ	Δ^2	Δ^3
1	24			
		96		
3	120		120	
		216		48
5	336		168	
		384		
7	720			

Here $h = 2$. With $x_0 = 1$, we have $x = 1 + 2p$ or $p = (x - 1)/2$. Substituting this value of p in Eq. (3.10), we obtain

$$y(x) = 24 + \frac{x-1}{2}(96) + \frac{\left(\frac{x-1}{2}\right)\left(\frac{x-1}{2}-1\right)}{2}(120) + \frac{\left(\frac{x-1}{2}\right)\left(\frac{x-1}{2}-1\right)\left(\frac{x-1}{2}-2\right)}{6}(48)$$

$$= x^3 + 6x^2 + 11x + 6.$$

To determine $y(8)$, we observe that $p = 7/2$. Hence, Eq. (3.10) gives:

$$y(8) = 24 + \frac{7}{2}(96) + \frac{(7/2)(7/2-1)}{2}(120) + \frac{(7/2)(7/2-1)(7/2-2)}{6}(48) = 990.$$

Direct substitution in $y(x)$ also yields the same value.

Note: This process of finding the value of y for some value of x *outside* the given range is called extrapolation and this example demonstrates the fact that if a tabulated function is a polynomial, then both interpolation and extrapolation would give exact values.

Example 3.5 Using Newton's forward difference formula, find the sum

$$S_n = 1^3 + 2^3 + 3^3 + \cdots + n^3.$$

We have

$$S_{n+1} = 1^3 + 2^3 + 3^3 + \cdots + n^3 + (n+1)^3$$

Hence

$$S_{n+1} - S_n = (n+1)^3,$$

or

$$\Delta S_n = (n+1)^3. \tag{i}$$

It follows that

$$\Delta^2 S_n = \Delta S_{n+1} - \Delta S_n = (n+2)^3 - (n+1)^3 = 3n^2 + 9n + 7,$$

$$\Delta^3 S_n = 3(n+1)^2 + 9n + 7 - (3n^2 + 9n + 7) = 6n + 12$$

$$\Delta^4 S_n = 6(n+1) + 12 - (6n + 12) = 6.$$

Since $\Delta^5 S_n = \Delta^6 S_n = \cdots = 0$, S_n is a fourth-degree polynomial in n.

Further,

$$S_1 = 1, \qquad \Delta S_1 = 8, \qquad \Delta^2 S_1 = 19, \qquad \Delta^3 S_1 = 18, \qquad \Delta^4 S_1 = 6.$$

Equation (3.10) gives

$$S_n = 1 + (n-1)(8) + \frac{(n-1)(n-2)}{2}(19) + \frac{(n-1)(n-2)(n-3)}{6}(18)$$

$$+ \frac{(n-1)(n-2)(n-3)(n-4)}{24}(6)$$

$$= \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2$$

$$= \left[\frac{n(n+1)}{2}\right]^2.$$

Example 3.6 Values of x (in degrees) and sin x are given in the following table:

x (in degrees)	sin x
15	0.2588190
20	0.3420201
25	0.4226183
30	0.5
35	0.5735764
40	0.6427876

Determine the value of sin 38°.

The difference table is

x	sin x	Δ	Δ^2	Δ^3	Δ^4	Δ^5
15	0.2588190					
		0.0832011				
20	0.3420201		–0.0026029			
		0.0805982		–0.0006136		
25	0.4226183		–0.0032165		0.0000248	
		0.0773817		–0.0005888		0.0000041
30	0.5		–0.0038053		0.0000289	
		0.0735764		–0.0005599		
35	0.5735764		–0.0043652			
		0.0692112				
40	0.6427876					

To find sin 38°, we use Newton's backward difference formula with $x_n = 40$ and $x = 38$. This gives

$$p = \frac{x - x_n}{h} = \frac{38 - 40}{5} = -\frac{2}{5} = -0.4.$$

Hence, using Eq. (3.14), we obtain

$$\begin{aligned} y(38) &= 0.6427876 - 0.4\,(0.0692112) + \frac{-0.4(-0.4+1)}{2}(-0.0043652) \\ &\quad + \frac{(-0.4)(-0.4+1)(-0.4+2)}{6}(-0.0005599) \\ &\quad + \frac{(-0.4)(-0.4+1)(-0.4+2)(-0.4+3)}{24}(0.0000289) \\ &\quad + \frac{(-0.4)(-0.4+1)(-0.4+2)(-0.4+3)(-0.4+4)}{120}(0.0000041) \\ &= 0.6427876 - 0.02768448 + 0.00052382 + 0.00003583 - 0.00000120 \\ &= 0.6156614. \end{aligned}$$

Example 3.7 Find the missing term in the following table:

x	y
0	1
1	3
2	9
3	–
4	81

Explain why the result differs from $3^3 = 27$.

Since four points are given, the given data can be approximated by a third degree polynomial in x. Hence $\Delta^4 y_0 = 0$. Substituting $\Delta = E - 1$ and simplifying, we get

$$E^4 y_0 - 4E^3 y_0 + 6E^2 y_0 - 4Ey_0 + y_0 = 0.$$

Since $E^r y_0 = y_r$, the above equation becomes

$$y_4 - 4y_3 + 6y_2 - 4y_1 + y_0 = 0.$$

Substituting for y_0, y_1, y_2 and y_4 in the above, we obtain

$$y_3 = 31.$$

The tabulated function is 3^x and the exact value of $y(3)$ is 27. The error is due to the fact that the exponential function 3^x is approximated by means of a polynomial in x of degree 3.

Example 3.8 The table below gives the values of tan x for $0.10 \le x \le 0.30$:

x	$y = \tan x$
0.10	0.1003
0.15	0.1511
0.20	0.2027
0.25	0.2553
0.30	0.3093

Find : (a) tan 0.12 (b) tan 0.26, (c) tan 0.40 and (d) tan 0.50.

The table of difference is

x	y	Δ	Δ^2	Δ^3	Δ^4
0.10	0.1003				
		0.0508			
0.15	0.1511		0.0008		
		0.0516		0.0002	
0.20	0.2027		0.0010		0.0002
		0.0526		0.0004	
0.25	0.2553		0.0014		
		0.0540			
0.30	0.3093				

(a) To find tan (0.12), we have $0.12 = 0.10 + p(0.05)$, which gives $p = 0.4$. Hence, Eq. (3.10) gives

$$\tan(0.12) = 0.1003 + 0.4(0.0508) + \frac{0.4(0.4-1)}{2}(0.0008) + \frac{0.4(0.4-1)(0.4-2)}{6}(0.0002) + \frac{0.4(0.4-1)(0.4-2)(0.4-3)}{24}(0.0002)$$

$$= 0.1205.$$

Chapter 4

Least Squares and Fourier Transforms

4.1 INTRODUCTION

In experimental work, we often encounter the problem of fitting a curve to data which are subject to errors. The strategy for such cases is to derive an approximating function that *broadly* fits the data without necessarily passing through the given points. The curve drawn is such that the discrepancy between the data points and the curve is least. In the method of least squares, the sum of the squares of the errors is minimized. For continuous functions, the method is discussed in Section 4.4.

The problem of approximating a function by means of Chebyshev polynomials is described in Section 4.5. This is important from the standpoint of digital computation.

In Chapter 3, we concentrated on polynomial interpolation, i.e., interpolation based on a linear combination of functions 1, x, x^2, …, x^n. On the other hand, trigonometric interpolation, i.e., interpolation based on trigonometric functions such as, cos x, sin x, cos $2x$, sin $2x$, … plays an important role in modelling vibrating systems. The Fourier series is a useful tool for dealing with periodic systems; but for aperiodic systems, the Fourier transform is the primary tool available. The computations of discrete Fourier transform and the Fast Fourier Transform (FFT) are discussed in detail in Section 4.6.

4.2 LEAST SQUARES CURVE FITTING PROCEDURES

Let the set of data points be (x_i, y_i), $i = 1, 2, \ldots, m$, and let the curve given by $Y = f(x)$ be fitted to this data. At $x = x_i$, the given ordinate is y_i and the

corresponding value on the fitting curve is $f(x_i)$. If e_i is the error of approximation at $x = x_i$, then we have

$$e_i = y_i - f(x_i) \tag{4.1}$$

If we write

$$\begin{aligned} S &= [y_1 - f(x_1)]^2 + [y_2 - f(x_2)]^2 + \cdots + [y_m - f(x_m)]^2 \\ &= e_1^2 + e_2^2 + \cdots + e_m^2, \end{aligned} \tag{4.2}$$

then the method of least squares consists in minimizing S, i.e., the sum of the squares of the errors. In the following sections, we shall study the linear and nonlinear least squares fitting to given data (x_i, y_i), $i = 1, 2, \ldots, m$.

4.2.1 Fitting a Straight Line

Let $Y = a_0 + a_1 x$ be the straight line to be fitted to the given data, viz. (x_i, y_i), $i = 1, 2, \ldots, m$. Then, corresponding to Eq. (4.2), we have

$$\begin{aligned} S = {} & [y_1 - (a_0 + a_1 x_1)]^2 + [y_2 - (a_0 + a_1 x_2)]^2 \\ & + \cdots + [y_m - (a_0 + a_1 x_m)]^2 \end{aligned} \tag{4.3}$$

For S to be minimum, we have

$$\begin{aligned} \frac{\partial S}{\partial a_0} = 0 = {} & -2[y_1 - (a_0 + a_1 x_1)] - 2[y_2 - (a_0 + a_1 x_2)] \\ & - \cdots - 2[y_m - (a_0 + a_1 x_m)] \end{aligned} \tag{4.4a}$$

and

$$\begin{aligned} \frac{\partial S}{\partial a_1} = 0 = {} & -2x_1[y_1 - (a_0 + a_1 x_1)] - 2x_2\,[y_2 - (a_0 + a_1 x_2)] \\ & - \cdots - 2x_m\,[y_m - (a_0 + a_1 x_m)] \end{aligned} \tag{4.4b}$$

The above equations simplify to

$$m a_0 + a_1(x_1 + x_2 + \cdots + x_m) = y_1 + y_2 + \cdots + y_m \tag{4.5a}$$

and

$$a_0(x_1 + x_2 + \cdots + x_m) + a_1\left(x_1^2 + x_2^2 + \cdots + x_m^2\right) = x_1 y_1 + x_2 y_2 + \cdots + x_m y_m \tag{4.5b}$$

or more compactly to

$$m a_0 + a_1 \sum_{i=1}^{m} x_i = \sum_{i=1}^{m} y_i \tag{4.6a}$$

and

$$a_0 \sum_{i=1}^{m} x_i + a_1 \sum_{i=1}^{m} x_i^2 = \sum_{i=1}^{m} x_i y_i \tag{4.6b}$$

Equations (4.6) are called the *normal equations*, and can be solved for a_0 and a_1, since x_i and y_i are known quantities.

We can obtain easily

$$a_1 = \frac{m\sum_{i=1}^{m} x_i y_i - \sum_{i=1}^{m} x_i \cdot \sum_{i=1}^{m} y_i}{m\sum_{i=1}^{m} x_i^2 - \left(\sum_{i=1}^{m} x_i\right)^2} \tag{4.7}$$

and then

$$a_0 = \bar{y} - a_1\bar{x}. \tag{4.8}$$

Since $\frac{\partial^2 S}{\partial a_0^2}$ and $\frac{\partial^2 S}{\partial a_1^2}$ are both positive at the points a_0 and a_1, it follows that these values provide a *minimum* of *S.* In Eq. (4.8), $\bar{x}$ and $\bar{y}$ are the means of x and y, respectively. From Eq. (4.8), we have

$$\bar{y} = a_0 + a_1\bar{x},$$

which shows that the fitted straight line passes through the centroid of the data points.

Sometimes, a goodness of fit is adopted. The correlation coefficient (cc) is defined as

$$\text{cc} = \sqrt{\frac{S_y - S}{S_y}}, \tag{4.9}$$

where

$$S_y = \sum_{i=1}^{m} (y_i - \bar{y})^2 \tag{4.10}$$

and S defined by Eq. (4.3).

If cc is close to 1, then the fit is considered to be good, although this is not always true.

Example 4.1 Find the best values of a_0 and a_1 if the straight line $Y = a_0 + a_1x$ is fitted to the data (x_i, y_i):

(1, 0.6), (2, 2.4), (3, 3.5), (4, 4.8), (5, 5.7)

Find also the correlation coefficient.

From the table of values given below, we find $\bar{x} = 3$, $\bar{y} = 3.4$, and

$$a_1 = \frac{5(63.6) - 15(17)}{5(55) - 225} = 1.26$$

Therefore,

$$a_0 = \bar{y} - a_1\bar{x} = -0.38.$$

x_i	y_i	x_i^2	$x_i y_i$	$(y_i - \bar{y})^2$	$(y_i - a_0 - a_1 x_i)^2$
1	0.6	1	0.6	7.84	0.0784
2	2.4	4	4.8	1.00	0.0676
3	3.5	9	10.5	0.01	0.0100
4	4.8	16	19.2	1.96	0.0196
5	5.7	25	28.5	5.29	0.0484
15	17.0	55	63.6	16.10	0.2240

The correlation coefficient $= \sqrt{\dfrac{16.10 - 0.2240}{16.10}} = 0.9930$.

Example 4.2 Certain experimental values of x and y are given below:

$$(0, -1),\ (2,\ 5),\ (5,\ 12),\ (7,\ 20)$$

If the straight line $Y = a_0 + a_1 x$ is fitted to the above data, find the approximate values of a_0 and a_1.

The table of values is given below.

x	y	x^2	xy
0	–1	0	0
2	5	4	10
5	12	25	60
7	20	49	140
14	36	78	210

The normal equations are

$$4a_0 + 14a_1 = 36$$

and

$$14a_0 + 78a_1 = 210$$

Solving the two equations, we obtain

$$a_0 = -1.1381 \quad \text{and} \quad a_1 = 2.8966$$

Hence the best straight line fit is given by

$$Y = -1.1381 + x(2.8966).$$

4.2.2 Multiple Linear Least Squares

Suppose that z is a linear function of two variables x and y. If the function $z = a_0 + a_1 x + a_2 y$ is fitted to the data (z_1, x_1, y_1), (z_2, x_2, y_2), $\ldots(z_m, x_m, y_m)$, then the sum

$$S = \sum_{i=1}^{m} (z_i - a_0 - a_1 x_i - a_2 y_i)^2$$

should be minimum. For this, we have

$$\frac{\partial S}{\partial a_0} = -2\sum(z_i - a_0 - a_1x_i - a_2y_i) = 0\,,$$

$$\frac{\partial S}{\partial a_1} = -2x_i\sum(z_i - a_0 - a_1x_i - a_2y_i) = 0\,,$$

and

$$\frac{\partial S}{\partial a_2} = -2y_i\sum(z_i - a_0 - a_1x_i - a_2y_i) = 0\,.$$

These equations simplify to

$$\left.\begin{aligned} ma_0 + a_1\Sigma x_i + a_2\Sigma y_i &= \Sigma z_i \\ a_0\Sigma x_i + a_1\Sigma x_i^2 + a_2\Sigma x_iy_i &= \Sigma z_ix_i \\ a_0\Sigma y_i + a_1\Sigma y_ix_i + a_2\Sigma y_i^2 &= \Sigma z_iy_i \end{aligned}\right\} \qquad (4.11)$$

from which a_0, a_1 and a_2 can be determined.

Example 4.3 Find the values of a_0, a_1 and a_2, so that the function $z = a_0 + a_1x + a_2y$ is fitted to the data (x, y, z) given below.

(0, 0, 2), (1, 1, 4), (2, 3, 3), (4, 2, 16) and (6, 8, 8).

We form the following table of values

x	y	z	x^2	xy	zx	y^2	yz
0	0	2	0	0	0	0	0
1	1	4	1	1	4	1	4
2	3	3	4	6	6	9	9
4	2	16	16	8	64	4	32
6	8	8	36	48	48	64	64
13	14	33	57	63	122	78	109

The normal equations are

$$5a_0 + 13a_1 + 14a_2 = 33$$
$$13a_0 + 57a_1 + 63a_2 = 122$$
$$14a_0 + 63a_1 + 78a_2 = 109$$

The solution of the above system is

$$a_0 = 2,\ a_1 = 5 \text{ and } a_2 = -3.$$

4.2.3 Linearization of Nonlinear Laws

The given data may not always follow a linear relationship. This can be ascertained from a plot of the given data. If a nonlinear model is to be fitted, it can be conveniently transformed to a linear relationship. Some nonlinear laws and their transformations are given as follows.

(a) $y = ax + \dfrac{b}{x}$

This can be written as

$$xy = ax^2 + b$$

Put $xy = Y$, $x^2 = X$. With these transformations, it becomes a linear model.

(b) $xy^a = b$

Taking logarithms of both sides, we get

$$\log_{10}x + a\log_{10}y = \log_{10}b.$$

In this case, we put

$$\log_{10}y = Y,\ \log_{10}x = X,$$

$$\frac{1}{a}\log_{10}b = A_0 \text{ and } -\frac{1}{a} = A_1,$$

so that

$$Y = A_0 + A_1X.$$

(c) $y = ab^x$

Taking logarithms of both sides, we obtain

$$\log_{10}y = \log_{10}a + x\log_{10}b$$
$$\Rightarrow Y = A_0 + A_1X,$$

where

$$Y = \log_{10}y,\ A_0 = \log_{10}a,$$
$$X = x, \text{ and } A_1 = \log_{10}b$$

(d) $y = ax^b$

We have

$$\log_{10}y = \log_{10}a + b\log_{10}x$$
$$\Rightarrow Y = A_0 + A_1X,$$

where

$$Y = \log_{10}y,\ A_0 = \log_{10}a,\ A_1 = b$$

and

$$X = \log_{10}x.$$

(e) $y = ae^{bx}$

In this case, we write

$$\ln y = \ln a + bx$$
$$\Rightarrow Y = A_0 + A_1X,$$

where

$$Y = \ln y,\ A_0 = \ln a,\ A_1 = b$$

and

$$X = x.$$

Example 4.4 Using the method of least squares, find constants a and b such that the function $y = ae^{bx}$ fits the following data:

(1.0, 2.473), (3.0, 6.722), (5.0, 18.274), (7.0, 49.673), (9.0, 135.026).

We have

$$y = ae^{bx}$$

Therefore,

$$\ln y = \ln a + bx$$
$$\Rightarrow Y = A_0 + A_1X,$$

where

$$Y = \ln y,\ A_0 = \ln a,\ A_1 = b \text{ and } X = x.$$

The table of values is given below

X	$Y = \ln y$	X^2	XY
1	0.905	1	0.905
3	1.905	9	5.715
5	2.905	25	14.525
7	3.905	49	27.335
9	4.905	81	44.145
25	14.525	165	92.625

We obtain

$$\bar{X} = 5,\ \bar{Y} = 2.905$$
$$A_1 = \frac{5(92.625) - 25(14.525)}{5(165) - 625} = 0.5 = b.$$

Then

$$A_0 = \bar{Y} - A_1\bar{X} = 2.905 - 0.5(5) = 0.405.$$

Hence,

$$a = e^{A_0} = e^{0.405} = 1.499.$$

It follows that the required curve is of the form

$$y = 1.499e^{0.5x}$$

Example 4.5 Using the method of least squares, fit a curve of the form $y = \dfrac{x}{a + bx}$ to the following data

(3, 7.148), (5, 10.231), (8, 13.509), (12, 16.434).

We have

$$y = \frac{x}{a + bx}$$
$$\Rightarrow \frac{1}{y} = \frac{a + bx}{x} = b + \frac{a}{x}$$
$$\Rightarrow Y = A_0 + A_1X,$$

where

$$A_0 = b,\ A_1 = a,\ X = \frac{1}{x} \text{ and } Y = \frac{1}{y}.$$

The table of values is

X	Y	X^2	XY
0.333	0.140	0.111	0.047
0.200	0.098	0.040	0.020
0.125	0.074	0.016	0.009
0.083	0.061	0.007	0.005
0.741	0.373	0.174	0.081

We obtain

$$A_1 = a = \frac{4(0.081) - 0.741(0.373)}{4(0.174) - (0.741)^2} = 0.324,\ \bar{X} = 0.185,\ \bar{Y} = 0.093$$

and $A_0 = b = \bar{Y} - a\bar{X} = 0.0331$.

Hence the required fit is $Y = 0.0331 + 0.324(X)$, which simplifies to

$$y = \frac{x}{0.324 + 0.0331(x)}.$$

[Note: The given data is obtained from the relation $y = \dfrac{x}{0.3162 + 0.0345x}$]

4.2.4 Curve Fitting by Polynomials

Let the polynomial of the nth degree,

$$Y = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n \tag{4.12}$$

be fitted to the data points (x_i, y_i), $i = 1, 2, \ldots, m$. We then have

$$\begin{aligned} S = &\left[y_1 - (a_0 + a_1x_1 + a_2x_1^2 + \cdots + a_nx_1^n)\right]^2 \\ &+ \left[y_2 - (a_0 + a_1x_2 + a_2x_2^2 + \cdots + a_nx_2^n)\right]^2 \\ &+ \cdots + \left[y_m - (a_0 + a_1x_m + a_2x_m^2 + \cdots + a_nx_m^n)\right]^2 \end{aligned} \tag{4.13}$$

Equating to zero the first partial derivatives and simplifying, we obtain the normal equations:

$$\left.\begin{aligned} ma_0 + a_1\Sigma x_i + a_2\Sigma x_i^2 + \cdots + a_n\Sigma x_i^n &= \Sigma y_i, \\ a_0\Sigma x_i + a_1\Sigma x_i^2 + \cdots + a_n\Sigma x_i^{n+1} &= \Sigma x_i y_i, \\ \vdots \\ a_0\Sigma x_i^n + a_1\Sigma x_i^{n+1} + \cdots + a_n\Sigma x_i^{2n} &= \Sigma x_i^n y_i \end{aligned}\right\} \tag{4.14}$$

where the summations are performed from $i = 1$ to $i = m$.

Chapter 6

Numerical Differentiation and Integration

6.1 INTRODUCTION

In Chapter 3, we were concerned with the general problem of interpolation, viz., given the set of values (x_0, y_0), (x_1, y_1), …, (x_n, y_n) of x and y, to find a polynomial $\phi(x)$ of the lowest degree such that $y(x)$ and $\phi(x)$ agree at the set of tabulated points. In the present chapter, we shall be concerned with the problems of numerical differentiation and integration. That is to say, given the set of values of x and y, as above, we shall derive formulae to compute:

(i) $\frac{dy}{dx}, \frac{d^2y}{dx^2}, \cdots$ for any value of x in $[x_0, x_n]$, and

(ii) $\int_{x_0}^{x_n} y\,dx$.

6.2 NUMERICAL DIFFERENTIATION

The general method for deriving the numerical differentiation formulae is to differentiate the interpolating polynomial. Hence, corresponding to each of the formulae derived in Chapter 3, we may derive a formula for the derivative. We illustrate the derivation with Newton's forward difference formula only, the method of derivation being the same with regard to the other formulae.

Consider Newton's forward difference formula:

$$y = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 + \cdots, \qquad (6.1)$$

where

$$x = x_0 + uh. \qquad (6.2)$$

Then

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx} = \frac{1}{h}\left(\Delta y_0 + \frac{2u-1}{2}\Delta^2 y_0 + \frac{3u^2 - 6u + 2}{6}\Delta^3 y_0 + \cdots\right). \qquad (6.3)$$

This formula can be used for computing the value of *dy*/*dx* for *non-tabular values* of *x*. For tabular values of *x*, the formula takes a simpler form, for by setting $x = x_0$ we obtain $u = 0$ from Eq. (6.2), and hence Eq. (6.3) gives

$$\left[\frac{dy}{dx}\right]_{x=x_0} = \frac{1}{h}\left(\Delta y_0 - \frac{1}{2}\Delta^2 y_0 + \frac{1}{3}\Delta^3 y_0 - \frac{1}{4}\Delta^4 y_0 + \cdots\right). \qquad (6.4)$$

Differentiating Eq. (6.3) once again, we obtain

$$\frac{d^2y}{dx^2} = \frac{1}{h^2}\left(\Delta^2 y_0 + \frac{6u-6}{6}\Delta^3 y_0 + \frac{12u^2 - 36u + 22}{24}\Delta^4 y_0 + \cdots\right), \qquad (6.5)$$

from which we obtain

$$\left[\frac{d^2y}{dx^2}\right]_{x=x_0} = \frac{1}{h^2}\left(\Delta^2 y_0 - \Delta^3 y_0 + \frac{11}{12}\Delta^4 y_0 + \cdots\right). \qquad (6.6)$$

Formulae for computing higher derivatives may be obtained by successive differentiation. In a similar way, different formulae can be derived by starting with other interpolation formuale. Thus,

(a) Newton's backward difference formula gives

$$\left[\frac{dy}{dx}\right]_{x=x_n} = \frac{1}{h}\left(\nabla y_n + \frac{1}{2}\nabla^2 y_n + \frac{1}{3}\nabla^3 y_n + \cdots\right) \qquad (6.7)$$

and

$$\left[\frac{d^2y}{dx^2}\right]_{x=x_n} = \frac{1}{h^2}\left(\nabla^2 y_n + \nabla^3 y_n + \frac{11}{12}\nabla^4 y_n + \frac{5}{6}\nabla^5 y_n + \cdots\right). \qquad (6.8)$$

(b) Stirling's formula gives

$$\left[\frac{dy}{dx}\right]_{x=x_0} = \frac{1}{h}\left(\frac{\Delta y_{-1} + \Delta y_0}{2} - \frac{1}{6}\frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2} + \frac{1}{30}\frac{\Delta^5 y_{-3} + \Delta^5 y_{-2}}{2} + \cdots\right) \qquad (6.9)$$

and

$$\left[\frac{d^2y}{dx^2}\right]_{x=x_0} = \frac{1}{h^2}\left(\Delta^2 y_{-1} - \frac{1}{12}\Delta^4 y_{-2} + \frac{1}{90}\Delta^6 y_{-3} - \cdots\right). \tag{6.10}$$

If a derivative is required near the end of a table, one of the following formulae may be used to obtain better accuracy

$$hy_0' = \left(\Delta - \frac{1}{2}\Delta^2 + \frac{1}{3}\Delta^3 - \frac{1}{4}\Delta^4 + \frac{1}{5}\Delta^5 - \frac{1}{6}\Delta^6 + \cdots\right)y_0 \tag{6.11}$$

$$= \left(\Delta + \frac{1}{2}\Delta^2 - \frac{1}{6}\Delta^3 + \frac{1}{12}\Delta^4 - \frac{1}{20}\Delta^5 + \frac{1}{30}\Delta^6 - \cdots\right)y_{-1} \tag{6.12}$$

$$h^2 y_0'' = \left(\Delta^2 - \Delta^3 + \frac{11}{12}\Delta^4 - \frac{5}{6}\Delta^5 + \frac{137}{180}\Delta^6 - \frac{7}{10}\Delta^7 + \frac{363}{560}\Delta^8 - \cdots\right)y_0 \tag{6.13}$$

$$= \left(\Delta^2 - \frac{1}{12}\Delta^4 + \frac{1}{12}\Delta^5 - \frac{13}{180}\Delta^6 + \frac{11}{180}\Delta^7 - \frac{29}{560}\Delta^8 + \cdots\right)y_{-1} \tag{6.14}$$

$$hy_n' = \left(\nabla + \frac{1}{2}\nabla^2 + \frac{1}{3}\nabla^3 + \frac{1}{4}\nabla^4 + \frac{1}{5}\nabla^5 + \frac{1}{6}\nabla^6 + \frac{1}{7}\nabla^7 + \frac{1}{8}\nabla^8 + \cdots\right)y_n \tag{6.15}$$

$$= \left(\nabla - \frac{1}{2}\nabla^2 - \frac{1}{6}\nabla^3 - \frac{1}{12}\nabla^4 - \frac{1}{20}\nabla^5 - \frac{1}{30}\nabla^6 - \frac{1}{42}\nabla^7 - \frac{1}{56}\nabla^8 - \cdots\right)y_{n+1}$$

(6.16)

$$h^2 y_n'' = \left(\nabla^2 + \nabla^3 + \frac{11}{12}\nabla^4 + \frac{5}{6}\nabla^5 + \frac{137}{180}\nabla^6 + \frac{7}{10}\nabla^7 + \frac{363}{560}\nabla^8 + \cdots\right)y_n \tag{6.17}$$

$$= \left(\nabla^2 - \frac{1}{12}\nabla^4 - \frac{1}{12}\nabla^5 - \frac{13}{180}\nabla^6 - \frac{11}{180}\nabla^7 - \frac{29}{560}\nabla^8 - \cdots\right)y_{n+1}. \tag{6.18}$$

For more details, the reader is referred to Interpolation and Allied Tables. The following examples illustrate the use of the formulae stated above.

Example 6.1 From the following table of values of x and y, obtain dy/dx and d^2y/dx^2 for $x = 1.2$:

x	y	x	y
1.0	2.7183	1.8	6.0496
1.2	3.3201	2.0	7.3891
1.4	4.0552	2.2	9.0250
1.6	4.9530		

The difference table is

x	y	Δ	Δ^2	Δ^3	Δ^4	Δ^5	Δ^6
1.0	2.7183						
		0.6018					
1.2	3.3201		0.1333				
		0.7351		0.0294			
1.4	4.0552		0.1627		0.0067		
		0.8978		0.0361		0.0013	
1.6	4.9530		0.1988		0.0080		0.0001
		1.0966		0.0441		0.0014	
1.8	6.0496		0.2429		0.0094		
		1.3395		0.0535			
2.0	7.3891		0.2964				
		1.6359					
2.2	9.0250						

Here $x_0 = 1.2,\ y_0 = 3.3201$ and $h = 0.2$. Hence Eq. (6.11) gives

$$\left[\frac{dy}{dx}\right]_{x=1.2} = \frac{1}{0.2}\left[0.7351 - \frac{1}{2}(0.1627) + \frac{1}{3}(0.0361) - \frac{1}{4}(0.0080) + \frac{1}{5}(0.0014)\right]$$

$$= 3.3205.$$

If we use formula (6.12), then we should use the differences diagonally downwards from 0.6018 and this gives

$$\left[\frac{dy}{dx}\right]_{x=1.2} = \frac{1}{0.2}\left[0.6018 + \frac{1}{2}(0.1333) - \frac{1}{6}(0.0294) + \frac{1}{12}(0.0067) - \frac{1}{20}(0.0013\right]$$

$$= 3.3205, \text{ as before.}$$

Similarly, formula (6.13) gives

$$\left[\frac{d^2y}{dx^2}\right]_{x=1.2} = \frac{1}{0.04}\left[0.1627 - 0.0361 + \frac{11}{12}(0.0080) - \frac{5}{6}(0.0014)\right] = 3.318.$$

Using formula (6.14), we obtain

$$\left[\frac{d^2y}{dx^2}\right]_{x=1.2} = \frac{1}{0.04}\left[0.1333 - \frac{1}{12}(0.0067) + \frac{1}{12}(0.0013)\right] = 3.32.$$

Example 6.2 Calculate the first and second derivatives of the function tabulated in the preceding example at the point $x = 2.2$ and also dy/dx at $x = 2.0$.

We use the table of differences of Example 6.1. Here $x_n = 2.2$, $y_n = 9.0250$ and $h = 0.2$. Hence formula (6.15) gives

$$\left[\frac{dy}{dx}\right]_{x=2.2} = \frac{1}{0.2}\left[1.6359 + \frac{1}{2}(0.2964) + \frac{1}{3}(0.0535) + \frac{1}{4}(0.0094) + \frac{1}{5}(0.0014)\right]$$

$$= 9.0228.$$

$$\left[\frac{d^2y}{dx^2}\right]_{x=2.2} = \frac{1}{0.04}\left[0.2964 + 0.0535 + \frac{11}{12}(0.0094) + \frac{5}{6}(0.0014)\right] = 8.992.$$

To find dy/dx at $x = 2.0$, we can use either (6.15) or (6.16). Formula (6.15) gives

$$\left[\frac{dy}{dx}\right]_{x=2.0} = \frac{1}{0.2}\left[1.3395 + \frac{1}{2}(0.2429) + \frac{1}{3}(0.0441) + \frac{1}{4}(0.0080) + \frac{1}{5}(0.0013) + \frac{1}{6}(0.0001)\right]$$

$$= 7.3896.$$

whereas from formula (6.16), we obtain

$$\left[\frac{dy}{dx}\right]_{x=2.0} = \frac{1}{0.2}\left[1.6359 - \frac{1}{2}(0.2964) - \frac{1}{6}(0.0535) - \frac{1}{12}(0.0094) - \frac{1}{20}(0.0014)\right]$$

$$= 7.3896.$$

Example 6.3 Find dy/dx and d^2y/dx^2 at $x = 1.6$ for the tabulated function of Example 6.1.

Choosing $x_0 = 1.6$, formula (6.9) gives

$$\left[\frac{dy}{dx}\right]_{x=1.6} = \frac{1}{0.2}\left(\frac{0.8978 + 1.0966}{2} - \frac{1}{2}\frac{0.0361 + 0.0441}{2} + \frac{1}{30}\frac{0.0013 + 0.0014}{2}\right)$$

$$= 4.9530.$$

Similarly, formula (6.10) yields

$$\left[\frac{d^2y}{dx^2}\right]_{x=1.6} = \frac{1}{0.04}\left[0.1988 - \frac{1}{12}(0.0080) + \frac{1}{90}(0.0001)\right] = 4.9525.$$

In the preceding examples, the tabulated function is e^x and hence it is easy to see that the error is considerably more in the case of the second derivatives. This is due to the reason that although the tabulated function and its approximating polynomial would agree at the set of data points, *their slopes at these points may vary considerably*. Numerical differentiation, is, therefore, an unsatisfactory process and should be used only in 'rare cases.' The next section will be devoted to a discussion of errors in the numerical differentiation formulae.

6.2.1 Errors in Numerical Differentiation

The numerical computation of derivatives involves two types of errors, viz. *truncation errors* and *rounding errors*. These are discussed below.

The truncation error is caused by replacing the tabulated function by means of an interpolating polynomial. This error can usually be estimated by formula (3.7). As noted earlier, this formula is of theoretical interest only, since, in practical computations, we usually do not have any information about the derivative $y^{(n+1)}(\xi)$. However, the truncation error in any numerical differentiation formula can easily be estimated in the following manner. Suppose that the tabulated function is such that its differences of a certain order are small and that the tabulated function is well approximated by the polynomial. (This means that the tabulated function does not have any rapidly varying components.) We know that 2ε is the total absolute error in the values of Δy_i, 4ε in the values of $\Delta^2 y_i$, etc., where ε is the absolute error in the values of y_i. Consider now, for example, Stirling's formula (6.9). This can be written in the form

$$\left[\frac{dy}{dx}\right]_{x=x_0} = \frac{\Delta y_{-1} + \Delta y_0}{2h} + T_1 = \frac{y_1 - y_{-1}}{2h} + T_1, \tag{6.19}$$

where T_1, the truncation error, is given by

$$T_1 = \frac{1}{6h}\left|\frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2}\right|. \tag{6.20}$$

Similarly, formula (6.10) can be written as

$$\left[\frac{d^2y}{dx^2}\right]_{x=x_0} = \frac{1}{h^2}\Delta^2 y_{-1} + T_2, \tag{6.21}$$

where

$$T_2 = \frac{1}{12h^2}|\Delta^4 y_{-2}|. \tag{6.22}$$

The *rounding error*, on the other hand, is inversely proportional to h in the case of first derivatives, inversely proportional to h^2 in the case of second

derivatives, and so on. Thus, *rounding error* increases as h decreases. Considering again Stirling's formula in the form of Eq. (6.19), the rounding error does not exceed $2\varepsilon/2h = \varepsilon/h$, where ε is the maximum error in the value of y_i. On the other hand, the formula

$$\left[\frac{dy}{dx}\right]_{x=x_0} = \frac{\Delta y_{-1} + \Delta y_0}{2h} - \frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{12h} + \cdots$$

$$= \frac{y_{-2} - 8y_{-1} + 8y_1 - y_2}{12h} + \cdots \tag{6.23}$$

has the maximum rounding error

$$\frac{18\varepsilon}{12h} = \frac{3\varepsilon}{2h}.$$

Finally, the formula

$$\left[\frac{d^2y}{dx^2}\right]_{x=x_0} = \frac{\Delta^2 y_{-1}}{h^2} + \cdots = \frac{y_{-1} - 2y_0 + y_1}{h^2} + \cdots \tag{6.24}$$

has the maximum rounding error $4\varepsilon/h^2$. It is clear that in the case of higher derivatives, the rounding error increases rather rapidly.

Example 6.4 Assuming that the function values given in the table of Example 6.1 are correct to the accuracy given, estimate the errors in the values of dy/dx and d^2y/dx^2 at $x = 1.6$.

Since the values are correct to 4D, it follows that $\varepsilon < 0.00005 = 0.5 \times 10^{-4}$.

Value of dy/dx at x = 1.6:

$$\text{Truncation error} = \frac{1}{6h}\left|\frac{\Delta^3 y_{-1} + \Delta^3 y_0}{2}\right|, \quad \text{from (6.20)}$$

$$= \frac{1}{6(0.2)}\frac{0.0361 + 0.0441}{2}$$

$$= 0.03342$$

and

$$\text{Rounding error} = \frac{3\varepsilon}{2h}, \quad \text{from (6.23)}$$

$$= \frac{3(0.5)10^{-4}}{0.4}$$

$$= 0.00038.$$

Hence,

$$\text{Total error} = 0.03342 + 0.00038 = 0.0338.$$

Using Stirling's formula from Eq. (6.19), with the first differences, we obtain

$$\left(\frac{dy}{dx}\right)_{x=1.6} = \frac{\Delta y_{-1} + \Delta y_0}{2h} = \frac{0.8978 + 1.0966}{0.4} = \frac{1.9944}{0.4} = 4.9860.$$

The *exact value* is 4.9530 so that the error in the above solution is (4.9860 – 4.9530), i.e., 0.0330, which agrees with the total error obtained above.

Value of d^2y/dx^2 *at* $x = 1.6$: Using Eq. (6.24), we obtain

$$\left[\frac{d^2y}{dx^2}\right]_{x=1.6} = \frac{\Delta^2 y_{-1}}{h^2} = \frac{0.1988}{0.04} = 4.9700$$

so that the error = 4.9700 – 4.9530 = 0.0170.

Also,

$$\textit{Truncation error} = \frac{1}{12h^2}|\Delta^4 y_{-2}| = \frac{1}{12(0.04)} \times 0.0080 = 0.01667$$

and

$$\textit{Rounding error} = \frac{4\varepsilon}{h^2} = \frac{4 \times 0.5 \times 10^{-4}}{0.04} = 0.0050.$$

Hence

$$\text{Total error in } \left[\frac{d^2y}{dx^2}\right]_{x=1.6} = 0.0167 + 0.0050 = 0.0217.$$

6.2.2 Cubic Spline Method

The cubic spline derived in Section 5.2 can conveniently be used to compute the *first* and *second* derivatives of a function. For a natural cubic spline, the recurrence formulae (5.31) or (5.32) may be used to compute the spline second derivatives depending upon the choice of the subdivisions. Then Eq. (5.29) gives the spline in the interval of interest, from which the first derivatives can be computed. For the first derivatives at the tabular points, it would, of course, be easier to use formulae (5.28) and (5.30) directly. If, on the other hand, end conditions involving the first derivatives are given, then recurrence formulae (5.34) or (5.36) may be used to compute the remaining first derivatives.

The following examples illustrate the use of the spline formulae in numerical differentiation.

Example 6.5 We consider the function $y(x) = \sin x$ in $[0, \pi]$.

Here $M_0 = M_N = 0$. Let $N = 2$, i.e., $h = \pi/2$. Then

$$y_0 = y_2 = 0, \quad y_1 = 1 \quad \text{and} \quad M_0 = M_2 = 0.$$

Using formulae (5.32), we obtain

$$M_0 + 4M_1 + M_2 = \frac{6}{h^2}(y_0 - 2y_1 + y_2)$$

or

$$M_1 = -\frac{12}{\pi^2}.$$

Formula (5.29) now gives the spline in each interval. Thus, in $0 \le x \le \pi/2$, we obtain

$$s(x) = \frac{2}{\pi}\left(\frac{-2x^3}{\pi^2} + \frac{3x}{2}\right),$$

which gives

$$s'(x) = \frac{2}{\pi}\left[-\frac{2}{\pi^2}(3x^2) + \frac{3}{2}\right]. \qquad \text{(i)}$$

Hence

$$s'\left(\frac{\pi}{4}\right) = \frac{2}{\pi}\left(-\frac{6}{\pi^2}\frac{\pi^2}{16} + \frac{3}{2}\right) = \frac{9}{4\pi} = 0.71619725.$$

Exact value of $s'(\pi/4) = \cos \pi/4 = 1/\sqrt{2} = 0.70710681.$ The percentage error in the computed value of $s'(\pi/4)$ is 1.28%. From (i),

$$s''(x) = -\frac{24}{\pi^3}x$$

and hence

$$s''\left(\frac{\pi}{4}\right) = -\frac{24}{\pi^3}\frac{\pi}{4} = -\frac{6}{\pi^2} = -0.60792710.$$

Since the exact value is $-1/\sqrt{2},$ the percentage error in this result is 14.03 %. We now consider values of $y = \sin x$ in intervals of 10° from $x = 0$ to $\pi.$ To obtain the spline second derivatives we used a computer and the results are given in the following table (up to $x = 90°$).

	$y''(x)$	
x (in degrees)	*Exact*	*Cubic spline*
10	−0.173 648 178	−0.174 089 426
20	−0.342 020 143	−0.342 889 233
30	−0.500 000 000	−0.501 270 524
40	−0.642 787 610	−0.644 420 964
50	−0.766 044 443	−0.767 990 999
60	−0.866 025 404	−0.868 226 016
70	−0.939 692 621	−0.942 080 425
80	−0.984 807 753	−0.987 310 197
90	−1.000 000 000	−1.002 541 048

It is seen that there is greater inaccuracy in the values of the spline second derivatives.

Example 6.6 From the following data for $y(x)$, find $y'(1.0)$.

x	–2	–1	2	3
$y(x)$	–12	–8	3	5

The function from which the above data was calculated is given by $y = -\frac{1}{15}x^3 - \frac{3}{20}x^2 + \frac{241}{60}x - 3.9$. Hence, the exact value of $y'(1)$ is 3.51667.

To apply the cubic spline formula (5.31), we observe that $h_1 = 1$, $h_2 = 3$ and $h_3 = 1$.

For $i = 1, 2$, the recurrence relation gives:

$$8M_1 + 3M_2 = -2$$

and

$$3M_1 + 8M_2 = -10,$$

since $M_0 = M_3 = 0$. We obtain $M_1 = \frac{14}{55}$ and $M_2 = -\frac{74}{55}$. In $-1 \le x \le 2$, we have

$$s_2(x) = \frac{1}{3}\left[\frac{(2-x)^3}{6}\cdot\frac{14}{55} + \frac{(x+1)^3}{6}\left(-\frac{74}{55}\right)\right]$$

$$+\frac{1}{3}\left[-8-\frac{21}{55}\right](2-x) + \frac{1}{3}\left[3-\frac{9}{6}\left(-\frac{74}{55}\right)\right](x+1)$$

Differentiating the above and putting $x = 1$, we obtain

$$y'(1) \approx s_2'(1.0) = \frac{1}{3}\left[-\frac{7}{55} - \frac{148}{55} + \frac{461}{55} + \frac{276}{55}\right]$$

$$= 3.52727, \text{ on simplification.}$$

6.2.3 Differentiation Formulae with Function Values

In Section 6.2, we developed forward, backward and central difference approximations of derivatives in terms of finite differences. From the computational point of view, it would be convenient to express the numerical differentiation formulae in terms of function values. We list below some differentiation formulae for use in numerical computations.

(i) *Forward Differences*

$$y'(x_i) = \frac{y_{i+1} - y_i}{h}; \qquad y'(x_i) = \frac{-y_{i+2} + 4y_{i+1} - 3y_i}{2h} + O(h^2)$$

$$y''(x_i) = \frac{y_i - 2y_{i+1} + y_{i+2}}{h^2}; \qquad y''(x_i) = \frac{-y_{i+3} + 4y_{i+2} - 5y_{i+1} + 2y_i}{h^2};$$

(ii) *Backward Differences*

$$y'(x_i) = \frac{y_i - y_{i-1}}{h}; \qquad y'(x_i) = \frac{3y_i - 4y_{i-1} + y_{i-2}}{2h};$$

$$y''(x_i) = \frac{y_i - 2y_{i-1} + y_{i-2}}{h^2}; \quad y''(x_i) = \frac{2y_i - 5y_{i-1} + 4y_{i-2} - y_{i-3}}{h^2};$$

(iii) *Central Differences*

$$y'(x_i) = \frac{y_{i+1} - y_{i-1}}{2h}; \qquad y'(x_i) = \frac{-y_{i+2} + 8y_{i+1} - 8y_{i-1} + y_{i-2}}{12h};$$

$$y''(x_i) = \frac{y_{i-1} - 2y_i + y_{i+1}}{h^2};$$

$$y''(x_i) = \frac{-y_{i+2} + 16y_{i+1} - 30y_i + 16y_{i-1} - y_{i-2}}{12h^2}$$

These formulae can be derived by using Taylor series expansion of the functions.

6.3 MAXIMUM AND MINIMUM VALUES OF A TABULATED FUNCTION

It is known that the maximum and minimum values of a function can be found by equating the first derivative to zero and solving for the variable. The same procedure can be applied to determine the maxima and minima of a tabulated function.

Consider Newton's forward difference formula

$$y = y_0 + p\Delta y_0 + \frac{p(p-1)}{2}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{6}\Delta^3 y_0 + \cdots$$

Differentiating this with respect to p, we obtain

$$\frac{dy}{dp} = \Delta y_0 + \frac{2p-1}{2}\Delta^2 y_0 + \frac{3p^2 - 3p + 2}{6}\Delta^3 y_0 + \cdots \tag{6.25}$$

For maxima or minima $dy/dp = 0$. Hence, terminating the right-hand side, for simplicity, after the third difference and equating it to zero, we obtain the quadratic for p

$$c_0 + c_1 p + c_2 p^2 = 0, \tag{6.26}$$

where

$$\left.\begin{aligned} c_0 &= \Delta y_0 - \frac{1}{2}\Delta^2 y_0 + \frac{1}{3}\Delta^3 y_0 \\ c_1 &= \Delta^2 y_0 - \Delta^3 y_0 \\ \text{and} \quad c_2 &= \frac{1}{2}\Delta^3 y_0. \end{aligned}\right\} \tag{6.27}$$

Values of x can then be found from the relation $x = x_0 + ph$.

Example 6.7 From the following table, find x, correct to two decimal places, for which y is maximum and find this value of y.

x	y
1.2	0.9320
1.3	0.9636
1.4	0.9855
1.5	0.9975
1.6	0.9996

The table of differences is

x	y	Δ	Δ^2
1.2	0.9320		
		0.0316	
1.3	0.9636		–0.0097
		0.0219	
1.4	0.9855		–0.0099
		0.0120	
1.5	0.9975		–0.0099
		0.0021	
1.6	0.9996		

Let $x_0 = 1.2$. Then formula (6.25), terminated after second differences, gives

$$0 = 0.0316 + \frac{2p-1}{2}(-0.0097)$$

from which we obtain $p = 3.8$. Hence

$$x = x_0 + ph = 1.2 + (3.8)\,(0.1) = 1.58.$$

For this value of x, Newton's backward difference formula at $x_n = 1.6$ gives

$$\begin{aligned} y(1.58) &= 0.9996 - 0.2\,(0.0021) + \frac{-0.2\,(-0.2+1)}{2}\,(-0.0099) \\ &= 0.9996 - 0.0004 + 0.0008 \\ &= 1.0. \end{aligned}$$

6.4 NUMERICAL INTEGRATION

The general problem of numerical integration may be stated as follows. Given a set of data points (x_0, y_0), (x_1, y_1), …, (x_n, y_n) of a function $y = f(x)$, where $f(x)$ is not known explicitly, it is required to compute the value of the definite integral

$$I = \int_a^b y\,dx. \tag{6.28}$$

As in the case of numerical differentiation, one replaces $f(x)$ by an interpolating polynomial $\phi(x)$ and obtains, on integration, an approximate value of the definite integral. Thus, different integration formulae can be obtained depending upon the type of the interpolation formula used. We derive in this section a general formula for numerical integration using Newton's forward difference formula.

Let the interval $[a, b]$ be divided into n equal subintervals such that $a = x_0 < x_1 < x_2 < \cdots x_n = b$. Clearly, $x_n = x_0 + nh$. Hence the integral becomes

$$I = \int_{x_0}^{x_n} y\, dx.$$

Approximating y by Newton's forward difference formula, we obtain

$$I = \int_{x_0}^{x_n} \left[y_0 + p\Delta y_0 + \frac{p(p-1)}{2}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{6}\Delta^3 y_0 + \cdots \right] dx.$$

Since $x = x_0 + ph, dx = h\, dp$ and hence the above integral becomes

$$I = h\int_0^n \left[y_0 + p\Delta y_0 + \frac{p(p-1)}{2}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{6}\Delta^3 y_0 + \cdots \right] dp,$$

which gives on simplification

$$\int_{x_0}^{x_n} y\, dx = nh\left[y_0 + \frac{n}{2}\Delta y_0 + \frac{n(2n-3)}{12}\Delta^2 y_0 + \frac{n(n-2)^2}{24}\Delta^3 y_0 + \cdots \right]. \quad (6.29)$$

From this *general formula*, we can obtain different integration formulae by putting n = 1, 2, 3, …, etc. We derive here a few of these formulae but it should be remarked that the trapezoidal and Simpson's 1/3-rules are found to give sufficient accuracy for use in practical problems.

6.4.1 Trapezoidal Rule

Setting n = 1 in the general formula (6.29), all differences higher than the first will become zero and we obtain

$$\int_{x_0}^{x_1} y\, dx = h\left(y_0 + \frac{1}{2}\Delta y_0 \right) = h\left[y_0 + \frac{1}{2}(y_1 - y_0) \right] = \frac{h}{2}(y_0 + y_1). \quad (6.30)$$

For the next interval $[x_1, x_2]$, we deduce similarly

$$\int_{x_1}^{x_2} y\, dx = \frac{h}{2}(y_1 + y_2) \quad (6.31)$$

and so on. For the last interval $[x_{n-1}, x_n]$, we have

$$\int_{x_{n-1}}^{x_n} y\,dx = \frac{h}{2}(y_{n-1} + y_n). \tag{6.32}$$

Combining all these expressions, we obtain the rule

$$\int_{x_0}^{x_n} y\,dx = \frac{h}{2}[y_0 + 2(y_1 + y_2 + \cdots + y_{n-1}) + y_n], \tag{6.33}$$

which is known as the *trapezoidal rule*.

The geometrical significance of this rule is that the curve $y = f(x)$ is replaced by n straight lines joining the points (x_0, y_0) and (x_1, y_1); (x_1, y_1) and (x_2, y_2), …, (x_{n-1}, y_{n-1}) and (x_n, y_n). The area bounded by the curve $y = f(x)$, the ordinates $x = x_0$ and $x = x_n$, and the x-axis is then approximately equivalent to the sum of the areas of the n trapeziums obtained.

The error of the trapezoidal formula can be obtained in the following way. Let $y = f(x)$ be continuous, well-behaved, and possess continuous derivatives in $[x_0, x_n]$. Expanding y in a Taylor's series around $x = x_0$, we obtain

$$\int_{x_0}^{x_1} y\,dx = \int_{x_0}^{x_1} \left[y_0 + (x - x_0)y_0' + \frac{(x - x_0)^2}{2} y_0'' + \cdots \right] dx$$

$$= hy_0 + \frac{h^2}{2} y_0' + \frac{h^3}{6} y_0'' + \cdots \tag{6.34}$$

Similarly,

$$\frac{h}{2}(y_0 + y_1) = \frac{h}{2}\left(y_0 + y_0 + hy_0' + \frac{h^2}{2} y_0'' + \frac{h^3}{6} y_0''' + \cdots \right)$$

$$= hy_0 + \frac{h^2}{2} y_0' + \frac{h^3}{4} y_0'' + \cdots \tag{6.35}$$

From Eqs. (6.34) and (6.35), we obtain

$$\int_{x_0}^{x_1} y\,dx - \frac{h}{2}(y_0 + y_1) = -\frac{1}{12} h^3 y_0'' + \cdots, \tag{6.36}$$

which is the error in the interval $[x_0, x_1]$. Proceeding in a similar manner we obtain the errors in the remaining subintervals, viz., $[x_1, x_2]$, $[x_2, x_3]$, … and $[x_{n-1}, x_n]$. We thus have

$$E = -\frac{1}{12}h^3(y_0'' + y_1'' + \cdots + y_{n-1}''), \tag{6.37}$$

where E is the *total error*. Assuming that $y''(\bar{x})$ is the largest value of the n quantities on the right-hand side of Eq. (6.37), we obtain

$$E = -\frac{1}{12}h^3 n y''(\bar{x}) = -\frac{b-a}{12}h^2 y''(\bar{x}) \tag{6.38}$$

since $nh = b - a$.

6.4.2 Simpson's 1/3-Rule

This rule is obtained by putting $n = 2$ in Eq. (6.29), i.e. by replacing the curve by $n/2$ arcs of second-degree polynomials or parabolas. We have then

$$\int_{x_0}^{x_2} y\,dx = 2h\left(y_0 + \Delta y_0 + \frac{1}{6}\Delta^2 y_0\right) = \frac{h}{3}(y_0 + 4y_1 + y_2).$$

Similarly,

$$\int_{x_2}^{x_4} y\,dx = \frac{h}{3}(y_2 + 4y_3 + y_4)$$

$$\vdots$$

and finally

$$\int_{x_{n-2}}^{x_n} y\,dx = \frac{h}{3}(y_{n-2} + 4y_{n-1} + y_n).$$

Summing up, we obtain

$$\int_{x_0}^{x_n} y\,dx = \frac{h}{3}[y_0 + 4(y_1 + y_3 + y_5 + \cdots + y_{n-1}) + 2(y_2 + y_4 + y_6 + \cdots + y_{n-2}) + y_n], \tag{6.39}$$

which is known as *Simpson's 1/3-rule*, or simply Simpson's rule. It should be noted that this rule requires the division of the whole range into an even number of subintervals of width h.

Following the method outlined in Section 6.4.1, it can be shown that the error in Simpson's rule is given by

$$\int_a^b y\,dx = \frac{h}{3}[y_0 + 4(y_1 + y_3 + y_5 + \cdots + y_{n-1}) + 2(y_2 + y_4 + y_6 + \cdots + y_{n-2}) + y_n]$$

$$= -\frac{b-a}{180}h^4 y^{\text{iv}}(\bar{x}), \tag{6.40}$$

where $y^{\text{iv}}(\bar{x})$ is the largest value of the fourth derivatives.

6.4.3 Simpson's 3/8-Rule

Setting $n = 3$ in Eq. (6.29), we observe that all the differences higher than the third will become zero and we obtain

$$\int_{x_0}^{x_3} y\,dx = 3h\left(y_0 + \frac{3}{2}\Delta y_0 + \frac{3}{4}\Delta^2 y_0 + \frac{1}{8}\Delta^3 y_0\right)$$

$$= 3h\left[y_0 + \frac{3}{2}(y_1 - y_0) + \frac{3}{4}(y_2 - 2y_1 + y_0) + \frac{1}{8}(y_3 - 3y_2 + 3y_1 - y_0)\right]$$

$$= \frac{3h}{8}(y_0 + 3y_1 + 3y_2 + y_3).$$

Similarly

$$\int_{x_3}^{x_6} y\,dx = \frac{3h}{8}(y_3 + 3y_4 + 3y_5 + y_6)$$

and so on. Summing up all these, we obtain

$$\int_{x_0}^{x_n} y\,dx = \frac{3h}{8}[(y_0 + 3y_1 + 3y_2 + y_3) + (y_3 + 3y_4 + 3y_5 + y_6) + \cdots$$

$$+ (y_{n-3} + 3y_{n-2} + 3y_{n-1} + y_n)]$$

$$= \frac{3h}{8}(y_0 + 3y_1 + 3y_2 + 2y_3 + 3y_4 + 3y_5 + 2y_6 + \cdots$$

$$+ 2y_{n-3} + 3y_{n-2} + 3y_{n-1} + y_n) \qquad (6.41)$$

This rule, called Simpson's (3/8)-rule, is not so accurate as Simpson's rule, the dominant term in the error of this formula being $-(3/80)\,h^5 y^{iv}(\bar{x})$.

6.4.4 Boole's and Weddle's Rules

If we wish to retain differences up to those of the fourth order, we should integrate between x_0 and x_4 and obtain Boole's formula

$$\int_{x_0}^{x_4} y\,dx = \frac{2h}{45}(7y_0 + 32y_1 + 12y_2 + 32y_3 + 7y_4) \qquad (6.42)$$

The leading term in the error of this formula can be shown to be

$$-\frac{8h^7}{945}y^{vi}(\bar{x}).$$

Chapter 7

Numerical methods

Numerical analysis is the area of mathematics and computer science that creates, analyzes, and implements algorithms for solving the problems of continuous mathematics numerically. Solving mathematical equations is an important requirement for various branches of science for several reasons. Many equations evade an analytic solution and in some cases the input could be numerical data from an experiment. Even when we have an analytical solution, for all practical purposes we need to evaluate the numeric value of the result, with the desired accuracy.

Numerical solution of systems of linear or non-linear equations, differential or integral equations, interpolation etc., are some of the major applications. In this book, we are not going in to the theoretical details but will focus on developing simple working programs to illustrate the concept. Detailed theory of numerical analysis can be found in text books.[1]Since the Taylor series and the polynomial interpolation forms the mathematical basis of many numerical methods, we start with a brief introduction of them.

It should be noted that almost all the example programs given in this chapter are for learning purpose only. They can be made much smaller and efficient by using library functions from numpy or scipy. You may cross check the results of you code with the output of standard library functions.

7.1 Taylor's Series

All of you are familiar with the sine function, starting from the high school days. You know how to differentiate it, integrate it and do so many other things with it. Now, how do we find the value of $sin(\theta)$, where $\theta = 10^0$, without using the tables or a calculator. The childhood friend becomes a bit unfamiliar at this point, I guess. But we know the value of $sin(\theta)$ and all it's derivatives at $\theta = 0$. Suppose we know the position, velocity and acceleration of a car at some point $t = t_0$, we can calculate it's position after a time interval Δt . The situation is similar in the case of $sin(10^0)$also, we use something called the Taylor series.

The Taylor series provides an algorithm for connecting the value of a function at two points to the derivatives that function. If a function and its derivatives are known at some point $x = x_0$, we can express $f(x)$ in the vicinity of that point using a polynomial. The Taylor series expansion is given by,

$$f(x) = f(x_0) + (x - x_0)f^{'}(x_0) + \frac{(x - x_0)^2}{2!}f^{''}(x_0) + \cdots + \frac{(x - x_0)^n}{n!}f^n(x_0) \tag{7.1}$$

[1] Introductory methods of numerical analysis by S.S.Sastry.

In the case of $f(x) = \sin(\theta)$, we know that $f(0) = 0$. The other terms involve the derivatives of $sin(x)$. The first, second and third derivatives of $sin(x)$ are $cos(x)$, $-sin(x)$ and $-cos(x)$, respectively. Evaluating each of these at zero, we get 1, 0 and -1 respectively. The terms with even powers vanish. taking $x_0 = 0$, results in,

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} + \cdots \quad = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \tag{7.2}$$

The program **sine-10deg.py** listed below calculates five terms to get the value

```
from numpy import sin

x = 10 * pi/180  # one degree in radians
sinx = 0
for n in range(3):
    term = (-1)**(n) * (x**(2*n+1)) / math.factorial(2*n+1)
    sinx = sinx + term
    err = sin(x) - sinx
    print (sinx,err)
```

The results are shown below. The difference with the built-in sine function at each step is printed. It can be seen that the error reduces with the number of terms. Modify the program to calculate for higher angles to observe the effect of accuracy. Also increase the number of terms to see the effect of it on accuracy.

0.17453292519943295 -0.0008847475325026166
0.17364682904373166 1.3486231986714614e-06
0.1736481786453548 -9.784244803245912e-10

7.2 Polynomial Interpolation

For any single valued and continuous function $y = f(x)$, we can calculate the value of y for any value of x in an interval $a < x < b$, where it is defined. Now, consider the problem that is just opposite of it.

We have a set of tabular values (x_0, y_0) (x_1, y_1) ... (x_i, y_i) satisfying a function $y = f(x)$ but we do not know what the function is. We construct a function $\phi(x)$ that satisfies all the given data points, obtained $\phi(x)$ may be only an approximation of $y = f(x)$. When a set of polynomials are used for constructing such a function like

$$y_n(x) = a_0 + a_1(x - x_0) + \cdots + a_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}) \tag{7.3}$$

is called polynomial interpolation. We can construct a polynomial of degree n from $n+1$ distinct data points. The coefficients a_i can be evaluated in the following manner. Since the polynomial passes through all the data points, when $x = x_0$, all the terms in 7.3 except a_0 should vanish due to the presence of $(x - x_0)$.

$$a_0 = y_o$$

Similarly for $x = x_1$, only the first two terms will be non-zero.

$$y_1 = a_0 + a_1(x_1 - x_0)$$

$$a_1 = \frac{(y_1 - y_0)}{(x_1 - x_0)}$$

and for $x = x_2$,

$$y_2 = a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)a_1(x_2 - x_1)$$

$$a_2 = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0}$$

This procedure can be continued to evaluate all the coefficients. Before proceeding with the general case, first let us consider a special case where the data points (x_i, y_i) are equally spaced. It is possible to construct difference tables and use them for interpolation.

7.2.1 Difference Table

For a set of given values (x_0, y_0) (x_1, y_1) ... (x_i, y_i) , the differences are
$\Delta y_0 = y_1 - y_0$
$\Delta y_1 = y_2 - y_1$
$\Delta y_2 = y_3 - y_2$ etc.
And
$\Delta^2 y_0 = \Delta y_1 - \Delta y_0 = (y_2 - y_1) - (y_1 - y_0)$
Similarly
$\Delta^3 y_0 = \Delta^2 y_1 - \Delta^2 y_0$
They can be expressed in the tabular form as shown below, to make manual computation easier. For numerical computation, we will use the data sets $[0, 2, 4, 6]$ and $[0, 8, 64, 216]$, that are derived from the equation $y = x^3$. This will help verification of the interpolated results by comparing with the analytic value.

x	y	Δ	Δ^2	Δ^3
0	0			
		8		
2	8		48	
		56		48
4	64		96	
		152		
6	216			

The Differences are available from the table. We can use Newton's formula for interpolation to calculate the value of y for an unknown value of x .

7.2.2 Newton's forward difference formula

On the equation

$$y_n(x) = a_0 + a_1(x - x_0) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1}) \tag{7.4}$$

applying the condition of equal spacing $x_1 = x_0 + h$, $x_2 = x_0 + 2h$ etc. results in

$$y_n(x) = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \cdots \tag{7.5}$$

where $p = \frac{x-x_0}{h}$ and $h = x_1 - x_0$. This is Newton's forward difference interpolation formula. Using it, let us calculate $y(x)$ for $x = 2.5$

$h = 2 - 0 = 2$

$p = \frac{(2.5-0)}{2} = 1.25$

$$y(2.5) = 0 + 1.25 \times 8 + \frac{1.25(1.25-1)48}{2} + \frac{1.25(1.25-1)(1.25-2)48}{6} = 15.625$$

The calculations could be done manually and the result agrees with the analytical result $2.5^3 = 15.625$. It is left as an exercise to the reader to use the function $y = x^4$ and repeat the exercise. The result would not agree, add one more point to observe the difference.

Next step is to write a Python program to do the same. The code should work for any number of input points. We need to create two lists to store the x and y values. The the table we can see that the differences require arrays of different sizes and the number of difference arrays depends on the number of data points. A better approach is to make a single difference array. After each iteration, we store the first element calculated in that iteration to a list, to be used in the interpolation formula. The program **forward-diff-table.py** listed below prints the forward difference table.

```
from copy import copy  #to make a deep copy of a list
x = [0,2,4,6]
y = [0,8,64,216]
n = len(x)
for a in x: print('%10.0f'%a, end = '')
print()
for a in y: print('%10.0f'%a, end = '')
print()
deltas = []   # list to store the required Delta values

diff = copy(y)
for k in range(0, n-1):
    tmp = copy(diff)
print((k+1)*5*' ', end = '') # adjust spacing
    for i in range(k,n-1):
        tmp[i+1] = (diff[i+1] - diff[i])
        print('%10.0f'%tmp[i+1], end = '')
    print()
    deltas.append(tmp[k+1])  # store delta
    diff = copy(tmp)
print ('Delta values ', deltas)
```

The algorithm works in the following manner. y is the starting value of difference array. Inside the loop the difference array is copied into a temporary array. Inside the inner loop, the next set of differences are calculated. The element required for future use is appended to a list. The output of the program is given below.

```
         0         2         4         6
         0         8        64       216
              8        56       152
                  48        96
                       48
Delta values  [0 8 48 48]
```

The code shown below uses the Delta to calculate the value of y for x=2.5.

```
nx = 2.5
h = x[1] - x[0]
p = (nx-x[0])/h
print('p = ', p)
ny = y[0] + p*deltas[0] + p*(p-1)*deltas[1]/2 + p*(p-1)*(p-2)*deltas[2]/6
print (ny)
```

Look for similarities between Taylor's series discussed in section 7.1 that and polynomial interpolation process. The derivative of a function represents an infinitesimal change in the function with respect to one of its variables. The finite difference is the discrete analog of the derivative. Using the divided difference method, we are in fact calculating the derivatives in the discrete form.

7.2.3 Newton's backward difference formula

By taking the final point as reference, the polynomial can be expressed as

$$y_n(x) = a_0 + a_1(x - x_n) + \cdots + a_n(x - x_n)(x - x_{n-1}) \cdots (x - x_1) \tag{7.6}$$

Applying similar conditions on this equation, we arrive at the Backward Difference Interpolation Formula.

$$y_n(x) = y_n + p\nabla y_n + \frac{p(p+1)}{2!}\nabla^2 y_n + \cdots + \frac{p(p+1)\ldots(p+n-1)}{n!}\nabla^n y_n \tag{7.7}$$

In this method also, the table is prepared in the same manner but it uses values from the end of the table. The program **backward-diff-table.py**, listed below, is identical to the previous program, except for statement deltas.append(tmp[-1]) that stores the last element from the kth difference array.

```
from copy import copy
x = [0,2,4,6]
y = [0,8,64,216]
n = len(x)
for a in x: print('%10.0f'%a, end = '')
print()
for a in y: print('%10.0f'%a, end = '')
print()
deltas = []   # list to store the required Delta values

diff = copy(y)
for k in range(0, n-1):
    tmp = copy(diff)
print((k+1)*5*' ', end = '') # adjust spacing
    for i in range(k,n-1):
        tmp[i+1] = (diff[i+1] - diff[i])
        print('%10.0f'%tmp[i+1], end = '')
    print()
    deltas.append(tmp[-1])  # store delta
    diff = copy(tmp)
print ('Delta values ',Delta)
```

The code given below can be used to verify the table.

```
nx = 2.5
h = x[1] - x[0]
p = (nx-x[-1])/h
print('p = ', p)
ny = y[-1] + p*deltas[0] + p*(p+1)*deltas[1]/2 + p*(p+1)*(p+2)*deltas[2]/6
print (ny, nx**3)
```

The program **diff-table-test.py** listed below plots the results from both the interpolations and the input data points along with $y = x^3$.

```
import numpy as np
import matplotlib.pyplot as plt
x = [0,2,4,6]
y = [0,8,64,216]
n = len(x)

nx = np.linspace(0,6,11)  # 11 values of x from 0 to 3
h = x[1] - x[0]
p = (nx-x[0])/h

deltas = [8, 48, 48]   #from difference table, forward
ny = y[0] + p*deltas[0] + p*(p-1)*deltas[1]/2 +
     p*(p-1)*(p-2)*deltas[2]/6
plt.plot(nx,ny, 'x', label='Forward')  # interpolated by FD method
p = (nx-x[-1])/h
deltas = [152, 96, 48]   #from difference table, backward
ny = y[-1] + p*deltas[0] + p*(p+1)*deltas[1]/2 + p*(p+1)*(p+2)*deltas[2]/6
plt.plot(nx,ny, '+', label='Backward')  # interpolated by BD method
plt.plot(x,y, 'o', label='Data points')    # given data points
plt.plot(nx, nx**3, label = '$y=x^3$')     # analytical value
plt.legend(framealpha=0.5)
plt.show()
```

The output of the program is given in figure 7.1

7.2.4 Lagrange's Interpolation formula

This method can be used for interpolating non-uniformly spaced data points. A set of tabular values (x_0, y_0) (x_1, y_1) ... (x_i, y_i) can be represented as

$$L_n(x) = \sum_{i=0}^{n} l_i(x) y_i$$

where $l_i(x)$ are polynomials in x of degree n.

$$l_i(x) = \frac{(x - x_0)(x - x_1)\ldots(x - x_{i-1})(x - x_{i+1})\ldots(x - x_n)}{(x_i - x_0)(x - x_1)\ldots(x_i - x_{i-1})(x_i - x_{i+1})\ldots(x_i - x_n)}$$

program **lagrange-poly.py** uses a non-uniformly spaced data set. We have generated this from equation $y = x^3$ to cross check the results of interpolation. The evaluation of the coefficients is simple, the i^{th}data point is excluded while calculating the ith term.

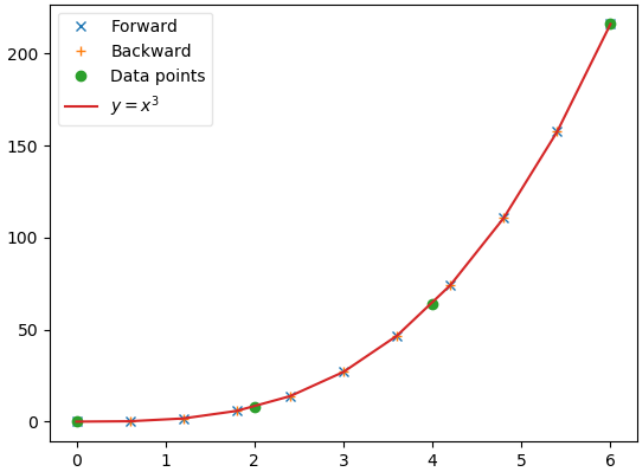


Figure 7.1: Newton's forward and backward interpolation results.

```
xa = [0, .5, 1.5, 3]
ya = [0, .125, 3.375, 27]
N = len(xa)

def lagra(x):
    res = 0.0
    for i in range(N):
        term = 1
        for k in range(N):
            if i != k:
                #print (x, xa[k], xa[i], xa[k])
                term = term * (x - xa[k]) / (xa[i]-xa[k])
        #print (term)
        res += term * ya[i]
    return res

print (lagra(2.5), 2.5**3)
```

The output is 15.625 15.625. The result tallies with the analytical value. Remove the comment to print each term.

7.2.5 Newton's General Interpolation Formula

Suppose the the given set is $(x_i, y_i), i = 0, 1 \ldots n-1$ and the polynomial is

$$y_n(x) = a_0 + a_1(x - x_0) + \cdots + a_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}) \tag{7.8}$$

We have already seen that the coefficients can be calculated from equations;

$$a_0 = y_o$$

$$a_1 = \frac{(y_1 - y_0)}{(x_1 - x_0)}$$

$$a_2 = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0}$$

They can be expressed better using the divided difference notation as shown below.

$$[y_0] = y_0$$

$$[y_0, y_1] = \frac{(y_1 - y_0)}{(x_1 - x_0)}$$

$$[y_0, y_1, y_3] = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} = \frac{[y_1, y_2] - [y_0, y_1]}{(x_2 - x_0)}$$

Using these notation, the polynomial can be re-written as;

$$\begin{aligned} y(x) &= [y_0] + [y_0, y_1](x - x_0) + [y_0, y_1, y_2](x - x_0)(x - x_1) + \\ &\quad \cdots + [y_0, \ldots, y_n](x - x_0)\ldots(x - x_{n-1}) \end{aligned} \tag{7.9}$$

The preparation of the forward difference table or equation 7.9 does not assume uniformly spaced data points. We will demonstrate it using a non-uniformly spaced data set, using the program **newton-general.py** listed below. Note the term $x(i-k)$ in the denominator, that decides the x interval. We have generated the input data using the equation $y = x^3$.

```
from copy import copy
x = [0, .5, 1.5, 3]
y = [0, .125, 3.375, 27]
n = len(x)

for a in x: print('%10.2f'%a, end= '')
print()
for a in y: print('%10.2f'%a, end = '')
print()
for k in range(0,n-1):
tmp = copy(y)
for i in range(k,n-1):
        tmp[i+1] = (y[i+1] - y[i]) / (x[i+1]-x[i-k])
        print('%10.2f'%tmp[i+1], end = '')
    print()
    y = copy(tmp)
print ('coefficients ',y)

#Verification
a = y
nx = 2.5
```

```
ny = a[0] + (nx - x[0])*a[1] + (nx - x[0])*(nx - x[1])*a[2] +
     (nx - x[0])*(nx - x[1])*(nx - x[2])*a[3]
print(ny, nx**3)
```

The program prints the difference table and the coefficients. We verify the validity of all these equations by calculating the value of y for any arbitrary value of x using equation 7.9and comparing it with x^3. Let us take $x = 2.5$ as an example.

$$y(2.5) = 0 + 1(2.5 - 0) + 3(2.5 - 0)(2.5 - 1) + 1(2.5 - 0)(2.5 - 1)(2.5 - 2) = 15.625$$

The result tallies with the exact value of $2.5^3 = 15.625$.

7.3 Numerical Integration

The general problem of numerical integration is to compute

$$I = \int_a^b f(x)\,dx \tag{7.10}$$

where a set of tabular values (x_0, y_0) (x_1, y_1) ... (x_i, y_i) satisfying a function $y = f(x)$ are given. To evaluate the integral 7.10, we divide it in to n equal intervals each having a width h. Then $x_n = x_0 + nh$ and $I = \int_{x_0}^{x_0+nh} f(x)\,dx$

According to Newton's forward difference formula,

$$f(x) = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \cdots$$

will lead to the Newton Cote's Quadrature formula[2],

$$I = nh\left[y_0 + \frac{n}{2}\Delta y_0 + \frac{n(2n-3)}{3!}\Delta^2 y_0 + \frac{n(n-1)^2}{4!}\Delta^3 y_0 + \ldots\right] \tag{7.11}$$

For different values of n this will result in different quadrature formulae.

7.3.1 Trapezoidal Rule

Truncating equation 7.11 at $n = 1$ results in Trapezoidal rule. The sum of the areas .

$$\int_{x_0}^{x_0+h} f(x)dx = h(y_0 + \frac{\Delta y_0}{2}) = h\left[y_0 + \frac{1}{2}(y_1 - y_0)\right] = \frac{h}{2}(y_0 + y_1)$$

Similarly

$$\int_{x_0+h}^{x_0+2h} ydx = \frac{h}{2}(y_1 + y_2)$$

Adding all the areas give

$$\int_{x_0}^{x_0+nh} f(x)dx = \frac{h}{2}\left[y_0 + 2(y_1 + y_2 + \ldots + y_{n-1}) + y_n\right] \tag{7.12}$$

[2] https://theengineeringmaths.com/wp-content/uploads/2017/11/num-diff-integ-web.pdf

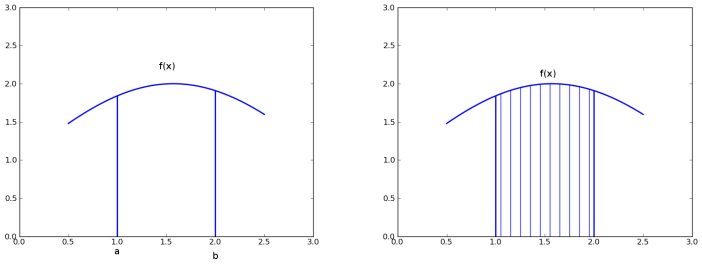


Figure 7.2: Area under the curve is divided it in to a large number of intervals. Area of each of them is calculated by assuming them to be trapezoids.

This is known as trapezoidal formula. The area under the curve, as shown in figure 7.2, is calculated by dividing it in to n section. Each section is a trapezoid in this case. The program *trapez.py* does integration of a given function using equation 7.12. We will choose an example where the results can be cross checked easily, the value of π is calculated by evaluating the area of a unit circle by integrating the equation of a circle.

Example **trapez.py**

```
from math import *
def y(x): # equation of a circle
    return sqrt(1.0 - x**2)

def trapez(f, a, b, n):
    h = (b-a) / n
    sum = 0
    x = 0.5 * h   # f(x) at middle of the slice
    for i in range (1,n):
        sum = sum + h * f(x)
        x = x + h
    return sum

print 4 * trapez(y, 0.0, 1.0,1000)
print 4 * trapez(y, 0.0, 1.0,10000)
```

The output is shown below. The result gets better by increasing n thus resulting in smaller h. .
3.140417031779045
3.141555466911023

7.3.2 Simpson's 1/3-Rule

Truncating equation 7.11 at $n = 2$ results in a second degree polynomial. The total curve will divided in to $\frac{n}{2}$ arcs , that also implies that n must be an even number. Area of sections will be given by,

$$\int_{x_0}^{x_2} ydx = 2h(y_0 + \Delta y_{0+} \frac{1}{6}\Delta^2 y_0) = \frac{h}{3}(y_0 + 4y_1 + y_2)$$

similarly

$$\int_{x_2}^{x_4} ydx = \frac{h}{3}(y_2 + 4y_3 + y_4)$$

etc. Summing all terms will give.

$$\int_{x_0}^{x_n} ydx = \frac{h}{3}\left[y + 4(y_1 + y_3 + \cdots + y_{n-1}) + 2(y_2 + y_4 + \cdots + y_{n-2}) + y_n\right] \tag{7.13}$$

This is known as Simpson's 1/3 rule. The program **simpson-1by3.py** calculates the area under an arc, one fourth of a circle, having unit radius. The equation of the circle $y = \sqrt{r^2 - x^2}$ is integrated from 0 to 1. The result can be easily cross checked with the expected value of $\frac{\pi}{4}$.

```
from math import *
def sqr(a):
return sqrt(1.0 - a**2)

def simp13(f, a, b, n):
    h = (b-a) / n
    result = f(a)    # first point
    for i in range (1,n,2):
        result = result + 4 * f(a + h * i)
    for i in range (2,n-1,2):
        result = result + 2 * f(a + h * i)
    result = result + f(b)  # last point
    return  h/3 * result

print(4*simp13(sqr,0.,1.,100))
print(4*simp13(sqr,0.,1.,1000))
```

The program prints

```
3.1411332053392274
3.1415781302139867
```

7.4 Derivatives from the Interpolation Formula

The Forward Difference Interpolation formula is expressed as

$$y_n(x) = a_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \cdots \tag{7.14}$$

where $p = \frac{x - x_0}{h}$ giving $\frac{dp}{dx} = \frac{1}{h}$

$$\frac{dy}{dp} = \Delta y_0 + \frac{(2p-1)}{2!}\Delta^2 y_0 + \frac{(3p^2 - 6p + 2)}{3!}\Delta^3 y_0 + \cdots \tag{7.15}$$

$$\frac{dy}{dx} = \frac{dy}{dp} \times \frac{dp}{dx}$$

$$\frac{dy}{dx} = \frac{1}{h}\left[\Delta y_0 + \frac{2p-1}{2!}\Delta^2 y_0 + \frac{(3p^2 - 6p + 2)}{3!}\Delta^2 y_0 + \ldots\right] \tag{7.16}$$

At $x = 0$, $p = 0$

$$\frac{dy}{dx} = \frac{1}{h}\left[\Delta y_0 + \frac{\Delta^2 y_0}{2!} + \frac{\Delta^3 y_0}{3!} + \dots\right] \tag{7.17}$$

To verify these equations again we use data generated from $y = x^3$. The program *fd-derivative-test.py* uses the results from the forward difference table and calculates the value of the derivative at $x = 1.5$ by using equation 7.16. The result is cross checked against $3x^2$, the derivative of x^3.

```
x = [0,1,2,3]
y = [0,1,8,27]
a = [0,1,3,1]   #coefficients from FD table

h = x[1] - x[0]
nx = 1.5   # new point
p = (nx-x[0])/h
dny = a[1] + (2*p-1)*a[2] + (3*p**2-6*p+2)*a[3]
print  (dny, 3*nx**2)
```

7.4.1 Numpy gradient function

The numpy module provides functions to differentiate data sets available in numerical form. Program **diff-data.py** loads two columns of data, x and y, from a file, calculates the first and second derivatives and plots them. The program is shown below.

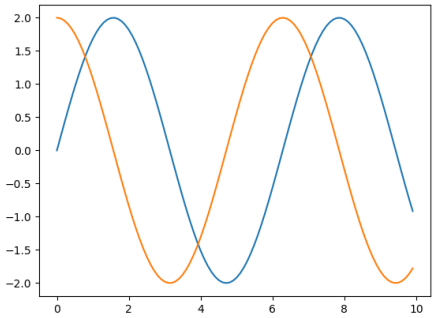
```
import numpy as np
import matplotlib.pyplot as plt
x,y = loadtxt("diff-data.txt", delimiter=' ')   # read 2 column data
dx = x[1] - x[0]
dy = np.diff(y)/dx
d2y = np.diff(dy)/dx
plt.subplot(1,3,1)
plt.plot(x,y)
plt.subplot(1,3,2)
plt.plot(x[:-1],dy)
plt.subplot(1,3,3)
plt.axis([0,10,0,20])
plt.plot(x[:-2],d2y)
plt.show()
```

Output of the program in shown in figure7.3.The input data looks like a parabola, first derivative has constant slope and the second derivative is a constant. The data is generated by the following lines of code.

```
t = linspace(0,10,11)
s = 5 * t**2    #S = ut + 1/2 at^2
savetxt('diff-data.txt', array([t,s]), delimiter = ' ')
```

7.5 First Order Ordinary Differential Equations

Differential equations are one of the most important mathematical tools used in producing models for physical and biological processes. In this section, we will discuss the numerical methods



7.7 Solution of Algebraic Equations

In general, an equation may have any number of roots, or no roots at all. For example $f(x) = x^2$ has a single root whereas $f(x) = sin(x)$ has an infinite number of roots. The roots can be located visually, by looking at the intersections with the x-axis. Another useful tool for detecting and bracketing roots is the incremental search method. The basic idea behind the incremental search method is simple: if $f(x1)$ and $f(x2)$ have opposite signs, then there is at least one root in the interval $(x1, x2)$. If the interval is small enough, it is likely to contain a single root. Thus the zeroes of $f(x)$ can be detected by evaluating the function at intervals of Δx and looking for change in sign.

There are several potential problems with the incremental search method: It is possible to miss two closely spaced roots if the search increment Δx is larger than the spacing of the roots. Certain singularities of $f(x)$ can be mistaken for roots. For example, $f(x) = tan(x)$ changes sign at odd multiples of $\pi/2$, but these locations are not true zeroes as shown in figure 7.6 (b).

Program **rootsearch.py** listed below implements the function *root*() that searches the roots of a function $f(x)$ from $x = a$ to $x = b$, increasing it by dx.

```
def func(x):
   return x**3-10.0*x*x + 5

def root(f,a,b,dx):
    x = a
    while True:
       f1 = f(x)
       f2 = f(x+dx)
       if f1*f2 < 0:
         return x, x + dx
       x = x + dx
       if x >=  b:
           return (None,None)

x,y = root(func, 0.0, 1.0,.1)
print x,y
```

The program finds that the root is between .7 and .8. Let us explore other methods like Bisection method, Newton-Raphson method etc., to find the root with a specified accuracy.

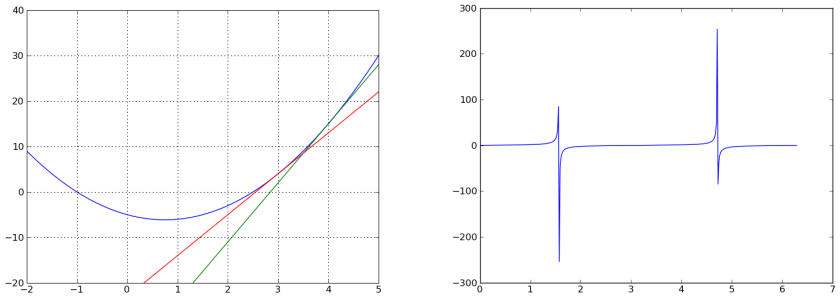


Figure 7.6: (a)Function $2x^2 - 3x - 5$ and its tangents at $x = 4$ and $x = 3$ (b) tan(x).

7.7.1 The Bisection method

The method of bisection finds the root by successively halving the interval until it becomes sufficiently small. Bisection is not the fastest method available for computing roots, but it is the most reliable. Once a root has been bracketed, bisection will always find it. The method of bisection works in the following manner. If there is a root between $x1$ and $x2$, then $f(x1) \times f(x2) < 0$. Next, we compute $f(x3)$, where $x3 = (x1 + x2)/2$. If $f(x2) \times f(x3) < 0$, then the root must be in $(x2, x3)$ and we replace the original bound $x1$ by $x3$. Otherwise, the root lies between $x1$ and $x3$, in that case $x2$ is replaced by $x3$. This process is repeated until the interval has been reduced to the specified value, say ε.

The number of bisections required to reach a prescribed limit, ε, is given by equation 7.22.

$$n = \frac{\ln(|\triangle x|)/\varepsilon}{\ln 2} \tag{7.22}$$

The program *bisection.py* finds the root of the equation $x^3 - 10x^2 + 5$. The starting values are found using the program *rootsearch.py*. The results are printed for two different values of accuracy.

Example **bisection.py**

```
import math def func(x):
       return x**3 - 10.0* x*x + 5

def bisect(f, x1, x2, epsilon=1.0e-9):
    f1 = f(x1)
    f2 = f(x2)
    if f1*f2 > 0.0:
       print 'x1 and x2 are on the same side of x-axis'
       return
    n = math.ceil(math.log(abs(x2 - x1)/epsilon)/math.log(2.0))
    n = int(n)
    for i in range(n):
        x3 = (x1 + x2) / 2.0   # take the midpoint
        f3 = f(x3)
        if f3 == 0.0:   # found the root
            return x3
        if f2*f3 < 0.0:  # root is in second half
```

```
                x1 = x3     # change start to midpoint
                f1 = f3
            else:
                x2 = x3     # root is in first half
                f2 = f3     # change end to midpoint
        return x3

x  = bisect(func, 0., 1., 1.0e-4)
print (x, func(x))
x  = bisect(func, .7, .8, 1.0e-4)
print (x, func(x))
x  = bisect(func, .7, .8, 1.0e-6)
print (x, func(x))
```

The output is shown below. Second column indicates amount of the error.

```
0.73455810546875 0.0005935349679475621
0.73466796875 -0.0008427397655319524
0.7346031188964843 5.08405033805559e-06
```

7.7.2 Regula Falsi (method of Chords)

In the bisection method the new interval depends only on the choice of end points, the function f(x) is not used for finding the next interval. A better approximation would be obtained by taking the straight line joining the points (a,f(a)) and (b,f(b)) intersecting the x-axis. This is given by

$$x = \frac{af(b) - bf(a)}{f(b) - f(a)}$$

The program **regula-falsi.py** implements this. Only one line of code is different from the code for bisection method. The line

```
x3 = (x1*f2 - x2*f1)/(f2-f1)
```

replaces

```
x3 = 0.5 * (x1 + x2)
```

The results after this change are

```
0.7346035077884963 1.0549783269198088e-11
0.7346035077893032 8.881784197001252e-16
0.7346035077893032 8.881784197001252e-16
```

It can be seen that the Regula Falsi is giving better results.

7.7.3 Newton-Raphson Method

The Newton–Raphson algorithm requires the derivative of the function also to evaluate the roots. Therefore, it is usable only in problems where $f'(x)$ can be readily computed. It does not require the value at two points to start with. We start with an initial guess which is reasonably close to the true root. Then the function is approximated by its tangent line and the x-intercept of the

tangent line is calculated. This value is taken as the next guess and the process is repeated. The Newton-Raphson formula is shown below.

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \tag{7.23}$$

Figure 7.6(a) shows the graph of the quadratic equation $2x^2 - 3x - 5 = 0$ and its two tangents. It can be seen that the zeros are at x = -1 and x = 2.5, and we use the program *newraph.py* shown below to find the roots. The function nr() is called twice, and we get the roots nearer to the corresponding starting values.

Example **newraph.py**

```
def f(x):
    return 2.0 * x**2 - 3*x - 5

def df(x):
    return 4.0 * x - 3

def nr(x, tol = 1.0e-9):
    for i in range(30):
        dx = -f(x)/df(x)
        #print x
        x = x + dx
        if abs(dx) < tol:
            return x

print nr(4)
print nr(0)
```

The output is shown below.

2.5
-1.0

Uncomment the print statement inside nr() to view how fast this method converges, compared to the bisection method. The program *newraph_plot.py*, listed below is used for generating the figure 7.6.

Example **newraph_plot.py**

```
import numpy as np
import matplotlib.pyplot as plt
def f(x):
      return 2.0 * x**2 - 3*x - 5

def df(x):
     return 4.0 * x - 3

vf = np.vectorize(f)
x = np.linspace(-2, 5, 100)
y = vf(x)
# Tangents at x=3 and 4, using one point slope formula
x1 = 4
```

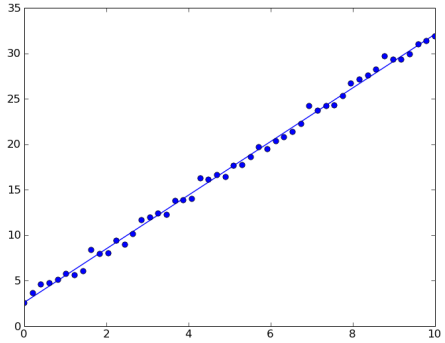


Figure 7.7: Output of lsfit.py

7.10 Least Squares Fitting

A mathematical procedure for finding the best-fitting curve $f(x)$ for a given set of points (x_n, y_n) by minimizing the sum of the squares of the vertical offsets of the points from the curve is called least squares fitting. The least square fit is obtained by minimizing the function,

$$S(a_0, a_1, \ldots, a_m) = \sum_{i=0}^{n} [y_i - f(x_i)]^2 \tag{7.24}$$

with respect to each $a_i$and the condition for that is

$$\frac{\partial S}{\partial a_i} = 0, \quad i = 0, 1, \ldots m \tag{7.25}$$

For a linear fit, the equation is

$$f(a, b) = a + bx$$

Solving the equations $\frac{\partial S}{\partial a} = 0$ and $\frac{\partial S}{\partial b} = 0$ will give the result,

$$b = \frac{\sum y_i(x - \overline{x})}{\sum x_i(x - \overline{x})}, \quad and \quad a = \overline{y} - \overline{x}b \tag{7.26}$$

where $\overline{x}$ and $\overline{y}$ are the mean values defined by the equations,

$$\overline{x} = \frac{1}{n+1} \sum_{i=0}^{n} x_i, \quad \overline{y} = \frac{1}{n+1} \sum_{i=0}^{n} y_i \tag{7.27}$$

The program lsfit.py demonstrates the usage of equations 7.26 and 7.27.

Example **lsfit.py**

```
import numpy as np
import matplotlib.pyplot as plt
NP = 50
r = 2* np.random.ranf([NP]) - 0.5
```

```
x = np.linspace(0,10,NP)
data = 3 * x + 2 + r

xbar = np.mean(x)
ybar = np.mean(data)
b = sum(data*(x-xbar)) / sum(x*(x-xbar))
a = ybar - xbar * b
print a,b
y = a + b * x
plt.plot(x,y)
plt.plot(x,data,'ob')
plt.show()
```

The raw data is made by adding random numbers (between -1 and 1) to the y coordinates generated by $y = 3 * x + 2$. The Numpy functions mean() and sum() are used. The output is shown in figure 7.7.

7.11 Monte Carlo methods

We only provide a glimpse of this method using an example program to calculate the value of π. We consider a square shaped area ranging from the coordinates (0,0) to (1,1), and a quarter of a circle centered at (0,0). We randomly generate points in the range 0 to 1, representing the x and y coordinates of a random point inside the square. We calculate the distance to the point from the origin to find out whether it falls inside the circle. This probability is decided by the area of the circle (one fourth of it) and that of the square. The program **pi-value.py** is listed below. Accuracy of the result depends on the number of trials.

```
from random import random
from math import sqrt

NP = 100000
inside = 0
for k in range(NP):
    x = random()
    y = random()
    r = sqrt(x**2 + y**2)
    if r <= 1:
        inside += 1

ratio = inside / NP
print (4*ratio)
```

7.12 Fourier Series

A Fourier series is an expansion of a periodic function $f(x)$ in terms of an infinite sum of sines and cosines. The computation and study of Fourier series is known as harmonic analysis and is extremely useful as a way to break up an arbitrary periodic function into a set of simple terms that can be plugged in, solved individually, and then recombined to obtain the solution to the original problem or an approximation to it to whatever accuracy is desired or practical.